

AWS Lambda

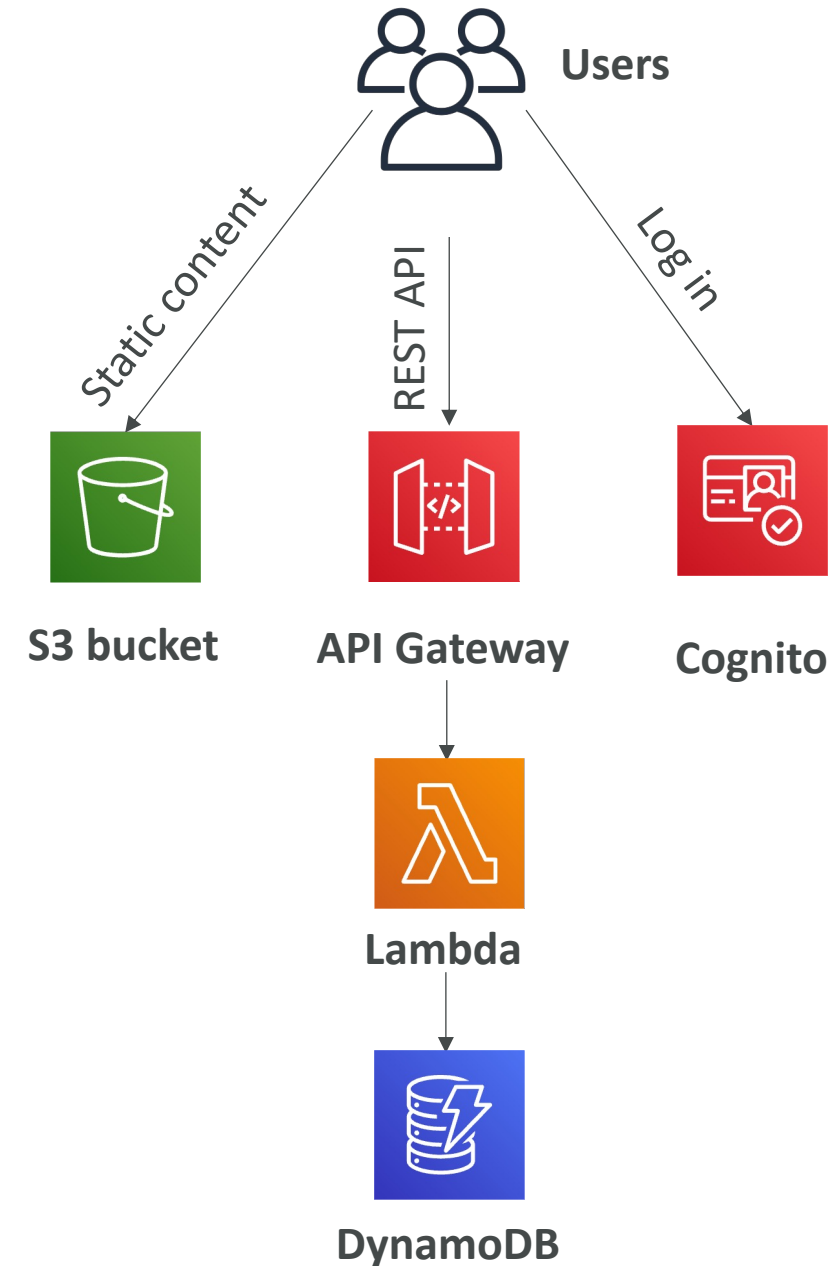
It's a serverless world

What's serverless?

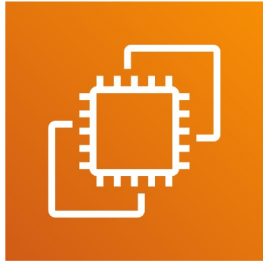
- Serverless is a new paradigm in which the developers don't have to manage servers anymore...
- They just deploy code
- They just deploy... functions !
- Initially... Serverless == FaaS (Function as a Service)
- Serverless was pioneered by AWS Lambda but now also includes anything that's managed: "databases, messaging, storage, etc."
- **Serverless does not mean there are no servers...**
it means you just don't manage / provision / see them

Serverless in AWS

- AWS Lambda
- DynamoDB
- AWS Cognito
- AWS API Gateway
- Amazon S3
- AWS SNS & SQS
- Aurora Serverless
- Step Functions
- Fargate



Why AWS Lambda



Amazon EC2

- Virtual Servers in the Cloud
- Limited by RAM and CPU
- Continuously running
- Scaling means intervention to add / remove servers



Amazon Lambda

- Virtual **functions** – no servers to manage!
- Limited by time - **short executions**
- Run **on-demand**
- **Scaling is automated!**

Benefits of AWS Lambda

- Easy Pricing:
 - Pay per request and compute time
 - Free tier of 1,000,000 AWS Lambda requests and 400,000 GBs of compute time
- Integrated with the whole AWS suite of services
- Integrated with many programming languages
- Easy monitoring through AWS CloudWatch
- Easy to get more resources per functions (up to 10GB of RAM!)
- Increasing RAM will also improve CPU and network!

AWS Lambda language support

- Node.js (JavaScript)
- Python
- Java
- C# (.NET Core) / Powershell
- Ruby
- Custom Runtime API (community supported, example Rust or Golang)
- Lambda Container Image
 - The container image must implement the Lambda Runtime API
 - ECS / Fargate is preferred for running arbitrary Docker images

AWS Lambda Integrations

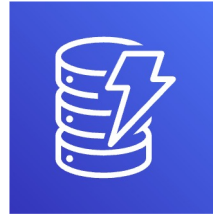
Main ones



API Gateway



Kinesis



DynamoDB



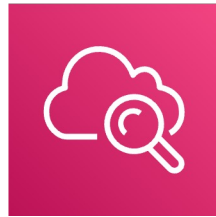
S3



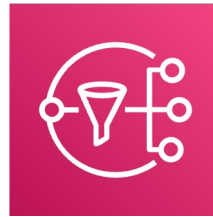
CloudFront



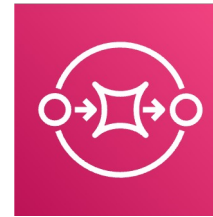
**CloudWatch Events
EventBridge**



CloudWatch Logs



SNS

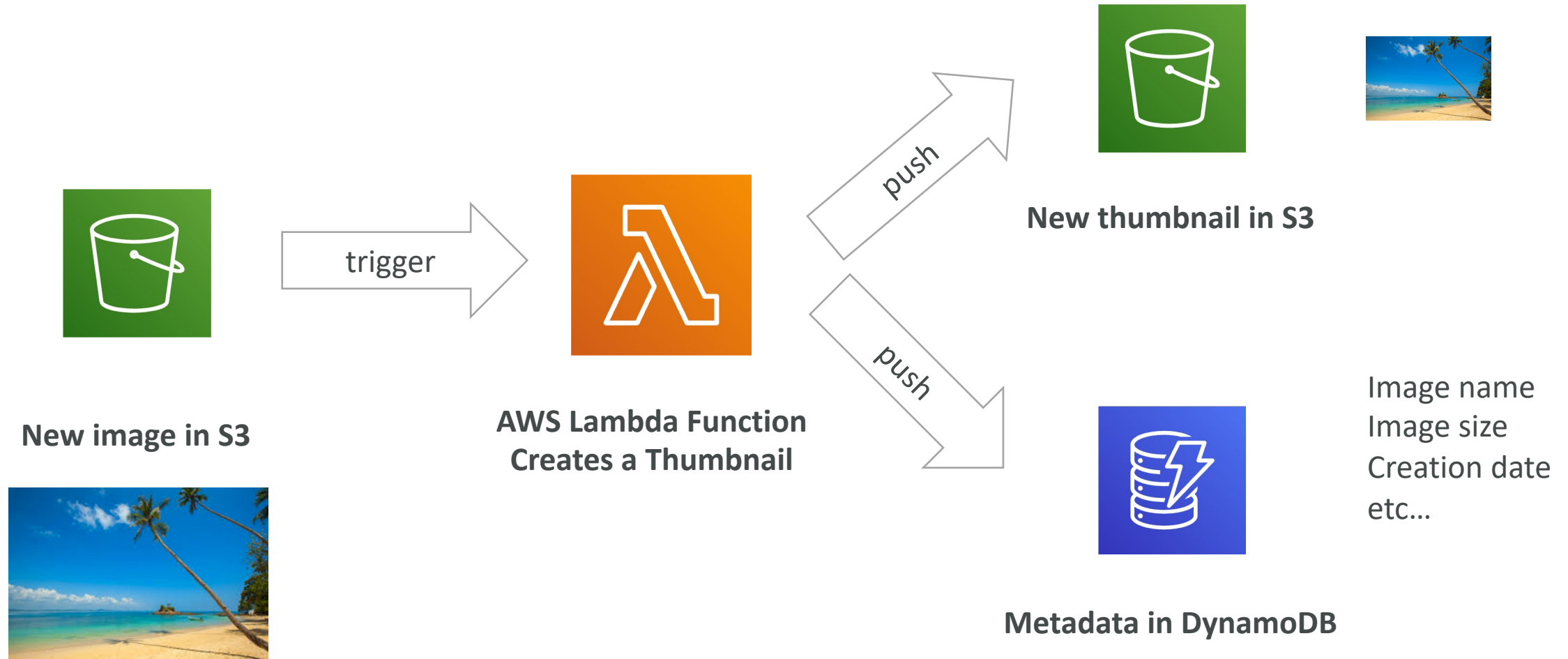


SQS



Cognito

Example: Serverless Thumbnail creation



Example: Serverless CRON Job

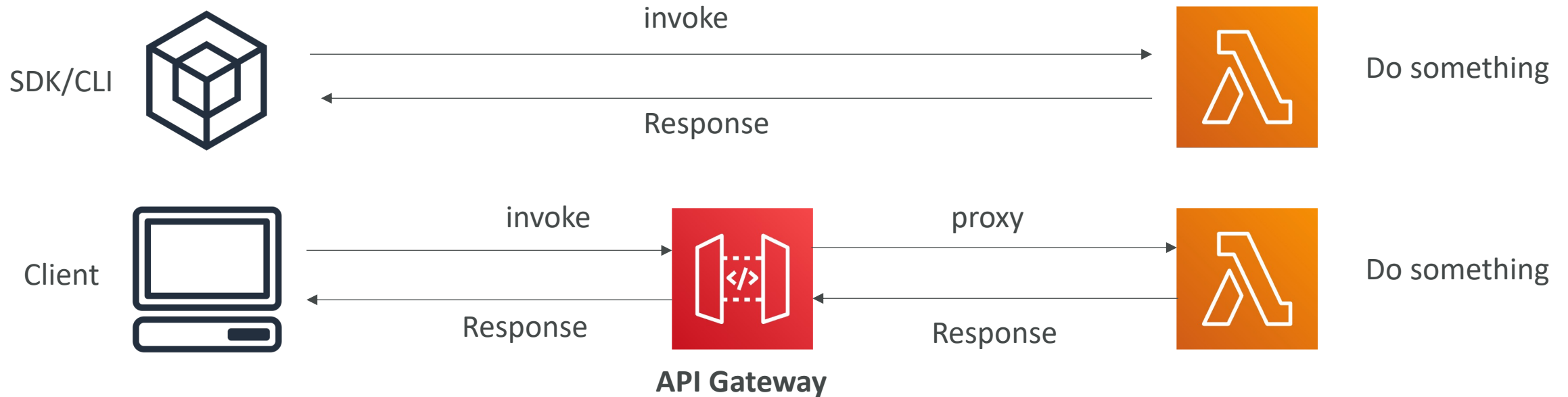


AWS Lambda Pricing: example

- You can find overall pricing information here:
<https://aws.amazon.com/lambda/pricing/>
- Pay per **calls**:
 - First 1,000,000 requests are free
 - \$0.20 per 1 million requests thereafter (\$0.0000002 per request)
- Pay per **duration**: (in increment of 1 ms)
 - 400,000 GB-seconds of compute time per month for FREE
 - == 400,000 seconds if function is 1 GB RAM
 - == 3,200,000 seconds if function is 128 MB RAM
 - After that \$1.00 for 600,000 GB-seconds
- It is usually very cheap to run AWS Lambda so it's very popular

Lambda – Synchronous Invocations

- Synchronous: CLI, SDK, API Gateway, Application Load Balancer
 - Results is returned right away
 - Error handling must happen client side (retries, exponential backoff, etc...)

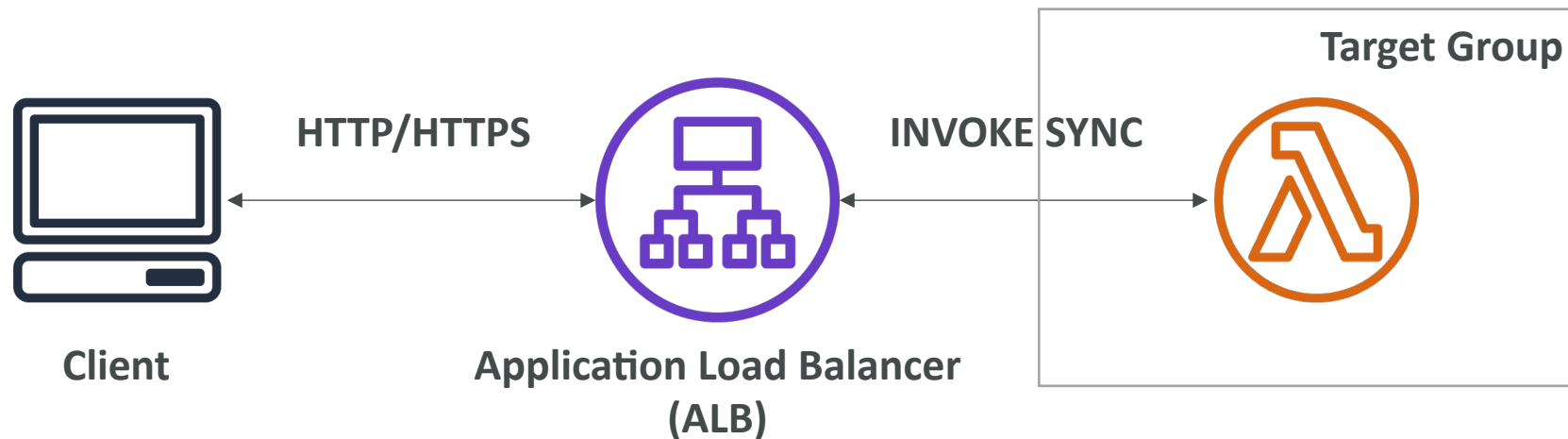


Lambda - Synchronous Invocations - Services

- User Invoked:
 - Elastic Load Balancing (Application Load Balancer)
 - Amazon API Gateway
 - Amazon CloudFront (Lambda@Edge)
 - Amazon S3 Batch
- Service Invoked:
 - Amazon Cognito
 - AWS Step Functions
- Other Services:
 - Amazon Lex
 - Amazon Alexa
 - Amazon Kinesis Data Firehose

Lambda Integration with ALB

- To expose a Lambda function as an HTTP(S) endpoint...
- You can use the **Application Load Balancer** (or an API Gateway)
- The Lambda function must be registered in a **target group**



ALB to Lambda: HTTP to JSON

Request Payload for Lambda Function

```
{
  "requestContext": {
    "elb": {
      "targetGroupArn": "arn:aws:elasticloadbalancing:us-east-2:123456789012:targetgroup/lambda-49e9d65c45c6791a"
    }
  },
  "httpMethod": "GET",
  "path": "/lambda",
  "queryStringParameters": {
    "query": "1234ABCD"
  },
  "headers": {
    "connection": "keep-alive",
    "host": "lambda-alb-123578498.us-east-2.elb.amazonaws.com",
    "user-agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36",
    "x-amzn-trace-id": "Root=1-5c536348-3d683b8b04734faae651f476",
    "x-forwarded-for": "72.12.164.125",
    "x-forwarded-port": "80",
    "x-forwarded-proto": "http"
  },
  "body": "",
  "isBase64Encoded": false
}
```

ELB information

HTTP Method & Path

Query String Parameters as Key/Value pairs

Headers as Key/Value pairs

Body (for POST, PUT...) & isBase64Encoded

Lambda to ALB conversions: JSON to HTTP

Response from the Lambda Function

```
{
  "statusCode": 200,
  "statusDescription": "200 OK",
  "headers": {
    "Content-Type": "text/html; charset=utf-8"
  },
  "body": "<h1>Hello world!</h1>",
  "isBase64Encoded": false
}
```

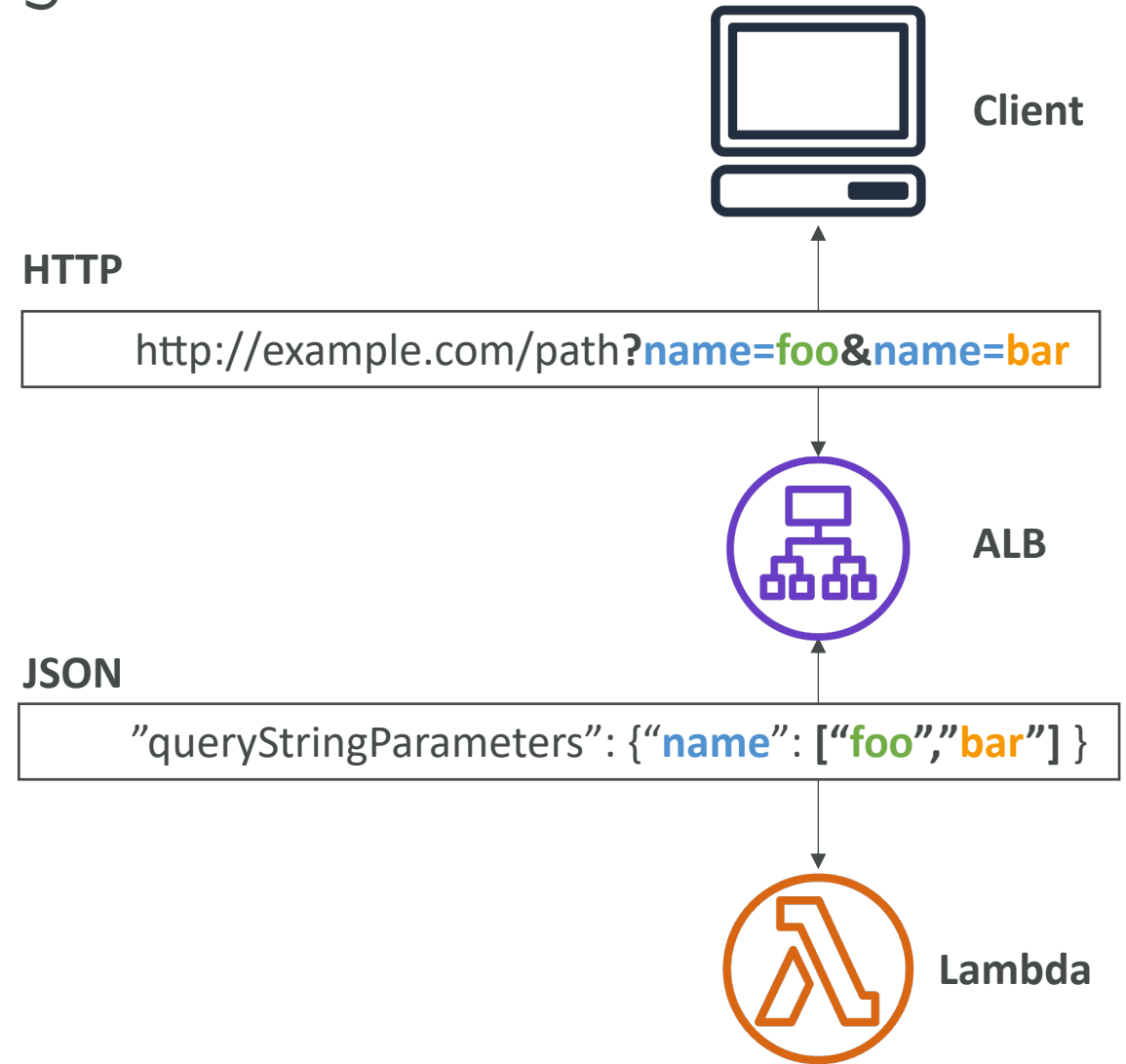
Status Code & Description

Headers as Key/Value pairs

Body & isBase64Encoded

ALB Multi-Value Headers

- ALB can support multi-value headers (ALB setting)
- When you enable multi-value headers, **HTTP headers and query string parameters** that are sent with **multiple values** are shown **as arrays** within the AWS Lambda event and response objects.



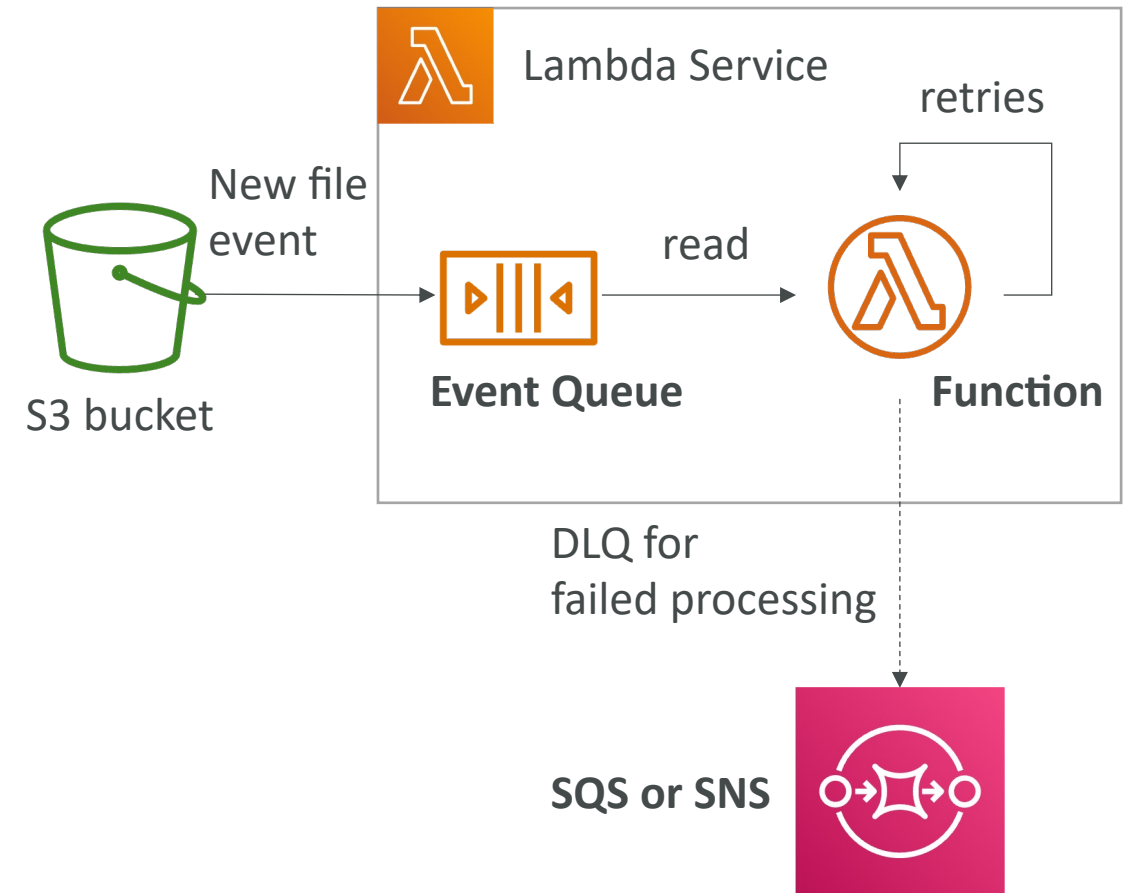
ALB + Lambda – Permissions

```
{  
  "Statement": {  
    "Effect": "Allow",  
    "Principal": {  
      "Service": "elasticloadbalancing.amazonaws.com"  
    },  
    "Action": "lambda:InvokeFunction",  
    "Resource": "arn:aws:lambda:us-west-2:123456789012:function:alb-function"  
  }  
}
```

Lambda Resource Policy

Lambda – Asynchronous Invocations

- S3, SNS, CloudWatch Events...
- The events are placed in an **Event Queue**
- Lambda attempts to retry on errors
 - 3 tries total
 - 1 minute wait after 1st, then 2 minutes wait
- Make sure the processing is **idempotent** (in case of retries)
- If the function is retried, you will see **duplicate logs entries in CloudWatch Logs**
- Can define a DLQ (dead-letter queue) – **SNS or SQS** – for failed processing (need correct IAM permissions)
- Asynchronous invocations allow you to speed up the processing if you don't need to wait for the result (ex: you need 1000 files processed)



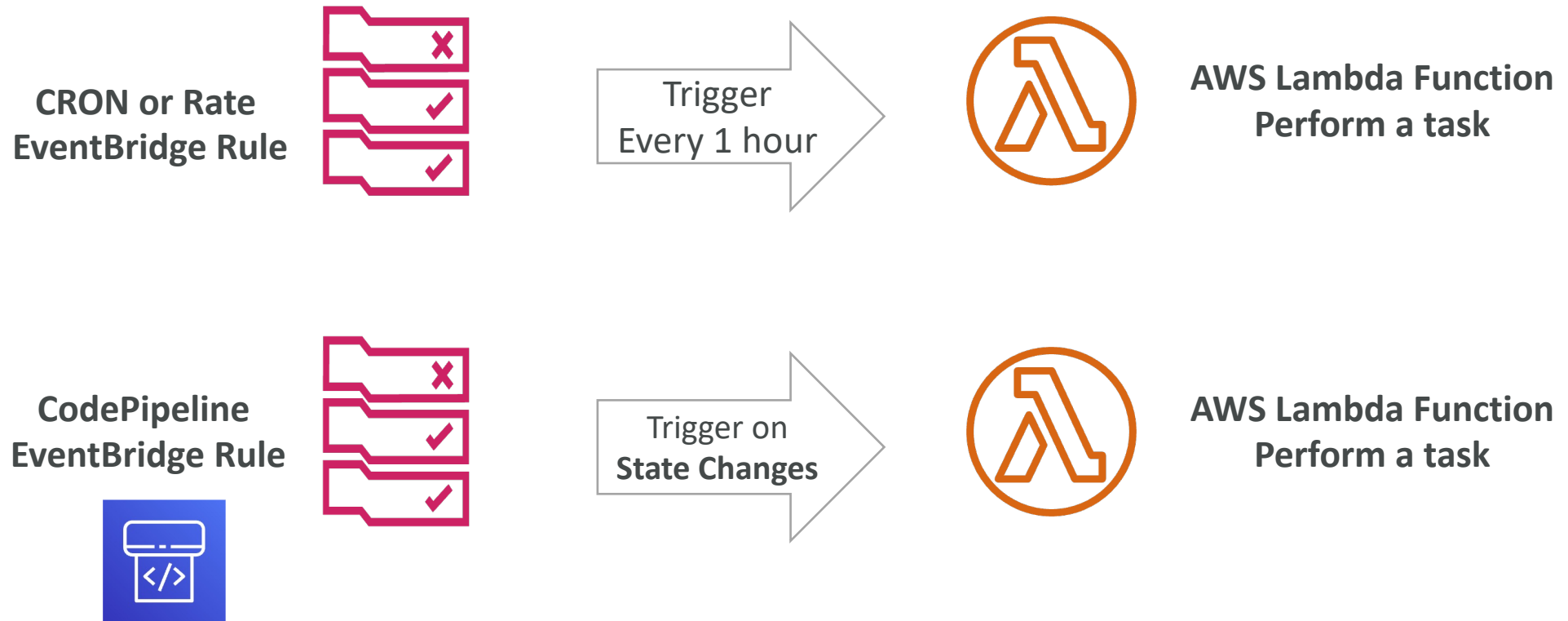
Lambda - Asynchronous Invocations - Services

- Amazon Simple Storage Service (S3)
- Amazon Simple Notification Service (SNS)
- Amazon CloudWatch Events / EventBridge
- AWS CodeCommit (CodeCommit Trigger: new branch, new tag, new push)
- AWS CodePipeline (invoke a Lambda function during the pipeline, Lambda must callback)

----- other -----

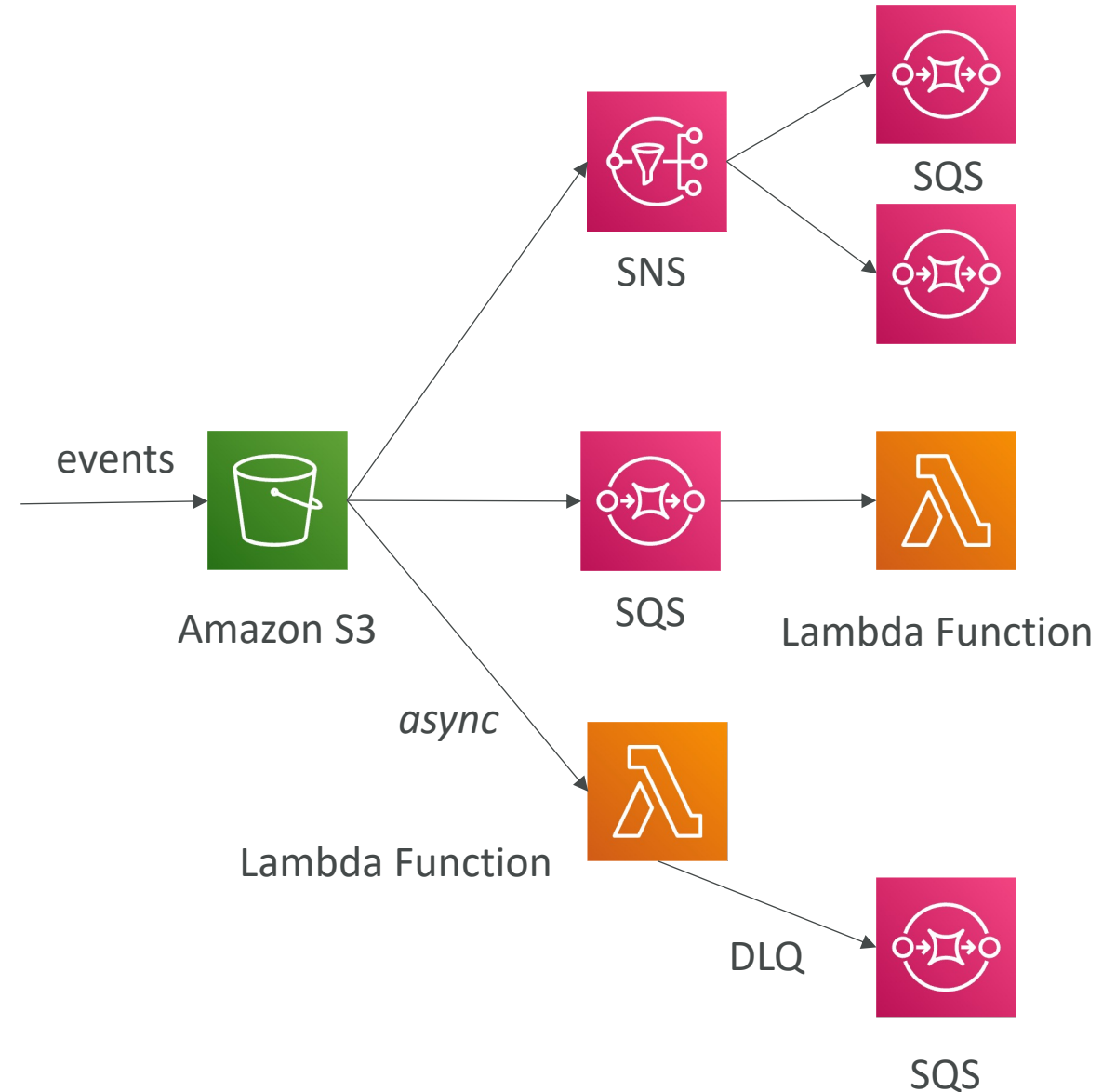
- Amazon CloudWatch Logs (log processing)
- Amazon Simple Email Service
- AWS CloudFormation
- AWS Config
- AWS IoT
- AWS IoT Events

CloudWatch Events / EventBridge

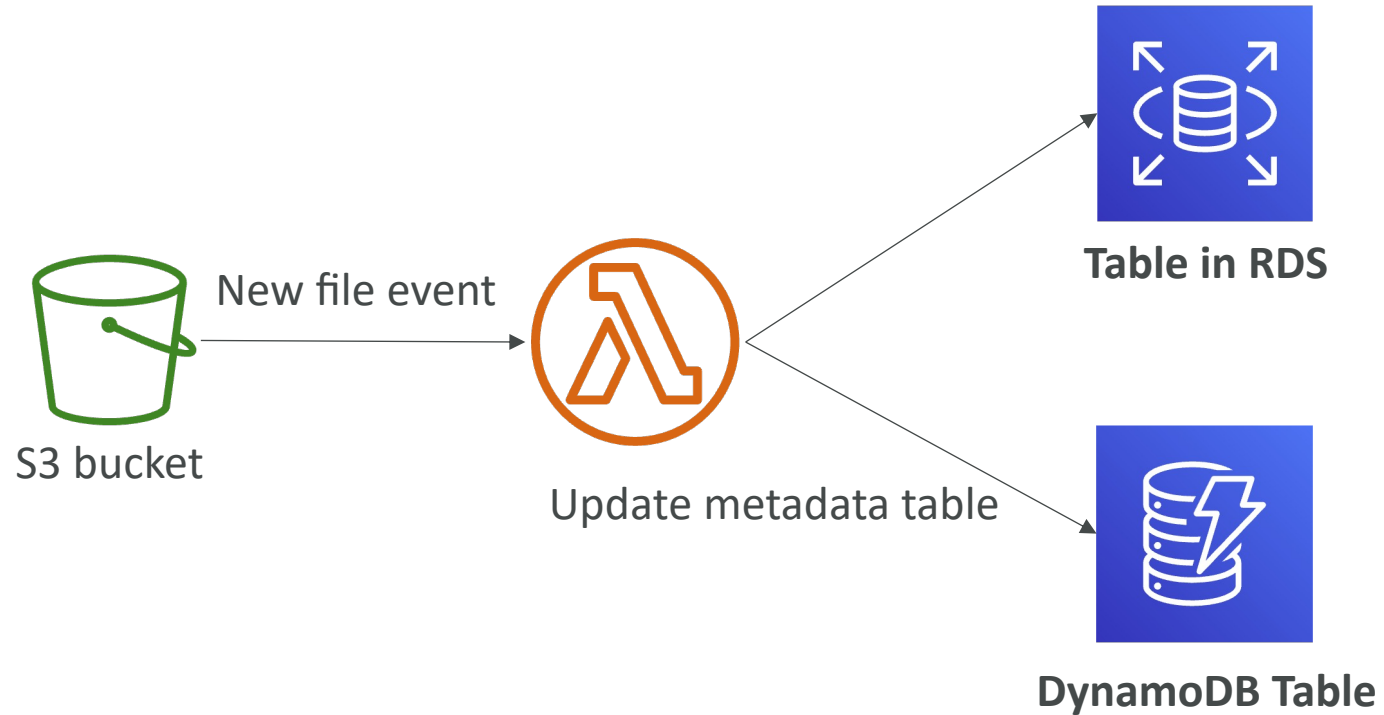


S3 Events Notifications

- S3:ObjectCreated, S3:ObjectRemoved, S3:ObjectRestore, S3:Replication...
- Object name filtering possible (*.jpg)
- Use case: generate thumbnails of images uploaded to S3
- S3 event notifications typically deliver events in seconds but can sometimes take a minute or longer
- If two writes are made to a single non-versioned object at the same time, it is possible that only a single event notification will be sent
- If you want to ensure that an event notification is sent for every successful write, you can enable versioning on your bucket.

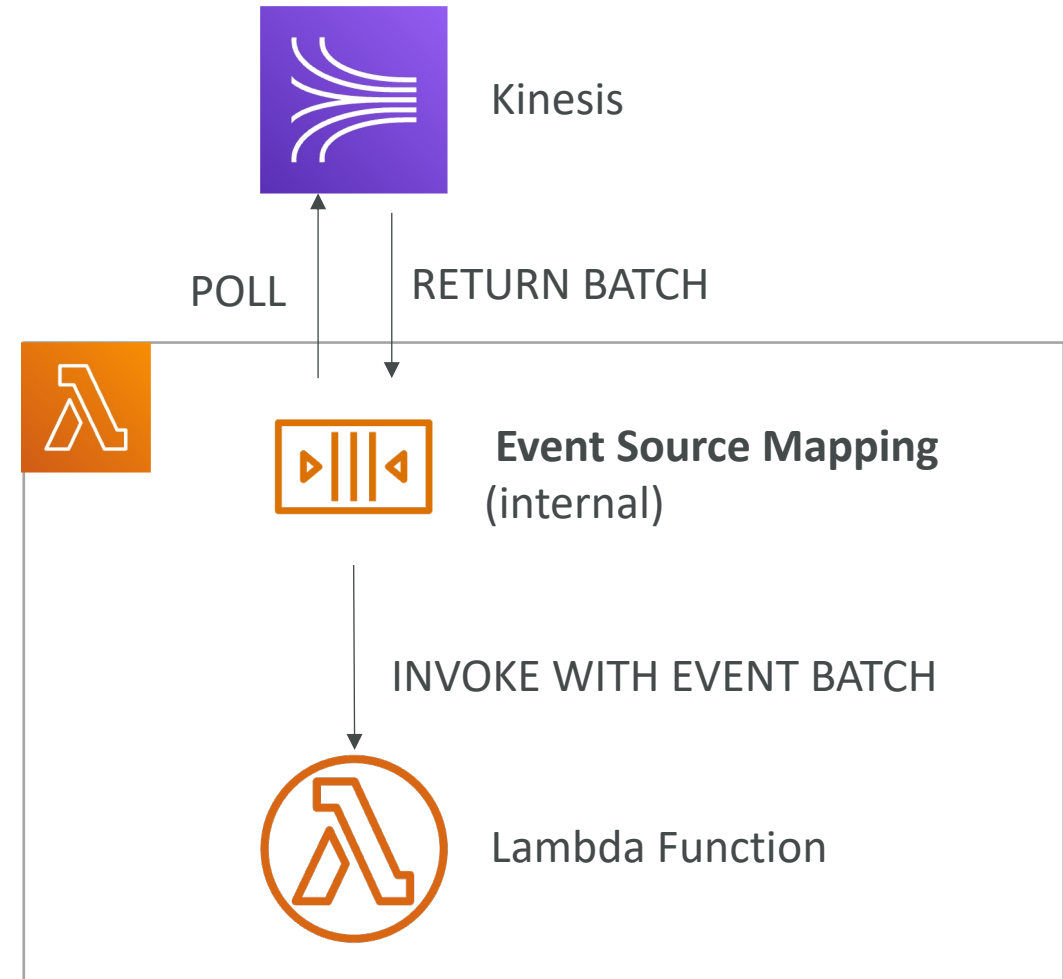


Simple S3 Event Pattern – Metadata Sync



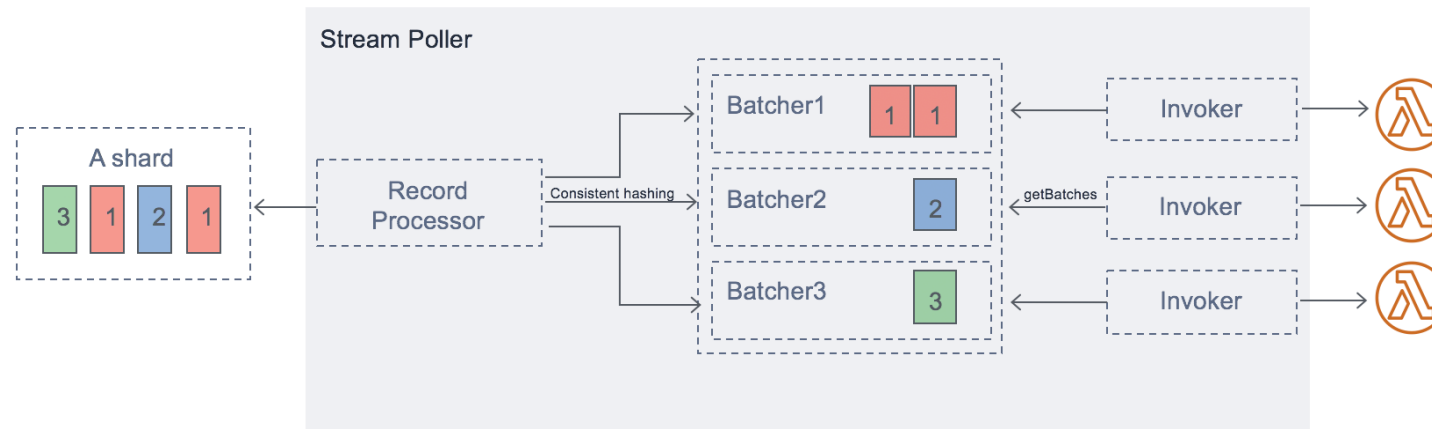
Lambda – Event Source Mapping

- Kinesis Data Streams
- SQS & SQS FIFO queue
- DynamoDB Streams
- Common denominator: records need to be polled from the source
- Your Lambda function is invoked synchronously



Streams & Lambda (Kinesis & DynamoDB)

- An event source mapping creates an iterator for each shard, processes items in order
- Start with new items, from the beginning or from timestamp
- Processed items aren't removed from the stream (other consumers can read them)
- Low traffic: use batch window to accumulate records before processing
- You can process multiple batches in parallel
 - up to 10 batches per shard
 - in-order processing is still guaranteed for each partition key,



<https://aws.amazon.com/blogs/compute/new-aws-lambda-scaling-controls-for-kinesis-and-dynamodb-event-sources/>

Streams & Lambda – Error Handling

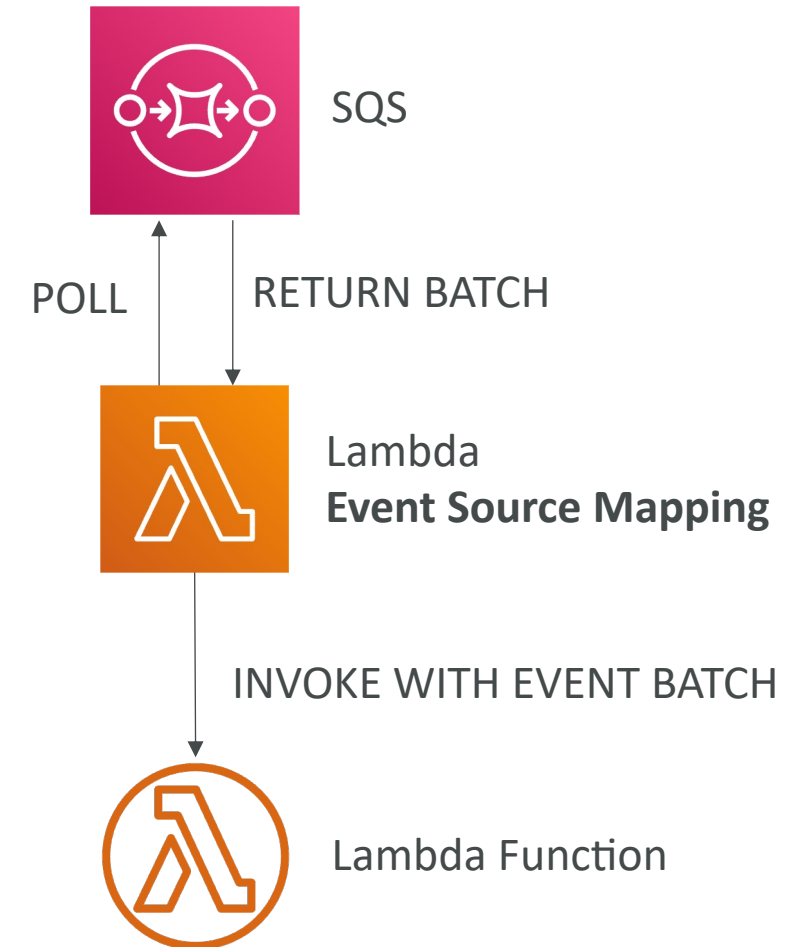
- By default, if your function returns an error, the entire batch is reprocessed until the function succeeds, or the items in the batch expire.
- To ensure in-order processing, processing for the affected shard is paused until the error is resolved
- You can configure the event source mapping to:
 - discard old events
 - restrict the number of retries
 - split the batch on error (to work around Lambda timeout issues)
- Discarded events can go to a **Destination**

Lambda – Event Source Mapping

SQS & SQS FIFO

- Event Source Mapping will poll SQS (**Long Polling**)
- Specify **batch size** (1-10 messages)
- Recommended: Set the queue visibility timeout to 6x the timeout of your Lambda function
- To use a DLQ
 - set-up on the SQS queue, not Lambda (DLQ for Lambda is only for async invocations)
 - Or use a Lambda destination for failures

Trigger configuration	
	SQS aws queue
SQS queue Choose an SQS queue to read messages from.	
arn:aws:sqs:us-east-1:160803060715:new.fifo	
Batch size The maximum number of messages to retrieve in a single batch.	
10	
In order to read from the SQS trigger, your execution role must have proper permissions.	
☒ Enable trigger Enable the trigger now, or create it in a disabled state for testing (recommended).	



Queues & Lambda

- Lambda also supports in-order processing for FIFO (first-in, first-out) queues, **scaling up to the number of active message groups**.
- For standard queues, items aren't necessarily processed in order.
- Lambda scales up to process a standard queue as quickly as possible.
- When an error occurs, batches are returned to the queue as individual items and might be processed in a different grouping than the original batch.
- Occasionally, the event source mapping might receive the same item from the queue twice, even if no function error occurred.
- Lambda deletes items from the queue after they're processed successfully.
- You can configure the source queue to send items to a dead-letter queue if they can't be processed.

Lambda Event Mapper Scaling

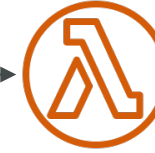
- Kinesis Data Streams & DynamoDB Streams:
 - One Lambda invocation per stream shard
 - If you use parallelization, up to 10 batches processed per shard simultaneously
- SQS Standard:
 - Lambda adds 60 more instances per minute to scale up
 - Up to 1000 batches of messages processed simultaneously
- SQS FIFO:
 - Messages with the same GroupID will be processed in order
 - The Lambda function scales to the number of active message groups

Lambda – Event and Context Objects

EventBridge



Lambda Function



invoke

```
{
  "version": "0",
  "id": "fe8d3c65-xmpl-c5c3-2c87-81584709a377",
  "detail-type": "RDS DB Instance Event",
  "source": "aws.rds",
  "account": "123456789012",
  "time": "2020-04-28T07:20:20Z",
  "region": "us-east-2",
  "resources": [
    "arn:aws:rds:us-east-2:123456789012:db:rdz6xmpliljlb1"
  ],
  "detail": {
    "EventCategories": [ "backup" ],
    "SourceType": "DB_INSTANCE",
    "SourceArn": "arn:aws:rds:us-east-2:123456789012:db:rdz6xmpliljlb1",
    "Date": "2020-04-28T07:20:20.112Z",
    "Message": "Finished DB Instance backup",
    "SourceIdentifier": "rdz6xmpliljlb1"
  }
}
```

Event Object

```
{
  "aws_request_id": "a3de505e-f16b-42f4-b3e6-bcd2e4a73903",
  "function_name": "MyFunction-AULC3LB8Q02F",
  "invoked_function_arn": "arn:aws:lambda:us-west-2:123456789012:function:MyFunction-AULC3LB8Q02F",
  "function_version": "$LATEST",
  "memory_limit_in_mb": "128",
  "log_group_name": "/aws/lambda/MyFunction-AULC3LB8Q02F",
  "log_stream_name": "2015/10/26/[$LATEST]c71058d852474b980f221f73402ad",
  "client_context": "None",
  ...
}
```

Context Object

Lambda – Event and Context Objects

- Event Object

- JSON-formatted document contains data for the function to process
- Contains information from the invoking service (e.g., EventBridge, custom, ...)
- Lambda runtime converts the event to an object (e.g., *dict* type in Python)
- Example: input arguments, invoking service arguments, ...

- Context Object

- Provides methods and properties that provide information about the invocation, function, and runtime environment
- Passed to your function by Lambda at runtime
- Example: `aws_request_id`, `function_name`, `memory_limit_in_mb`, ...

Lambda – Event and Context Objects

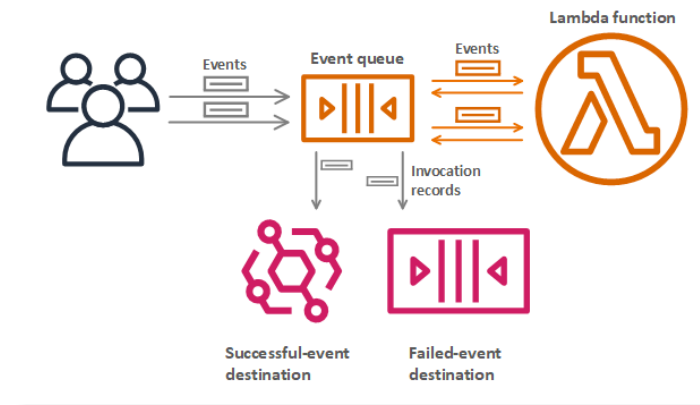
Access Event & Context Objects using Python

```
def lambda_handler(event, context):  
    print("Event Source:", event.source)  
    print("Event Region:", event.region)  
  
    print("Lambda Request ID:", context.aws_request_id)  
    print("Lambda function ARN:", context.function_name)  
    print("Lambda function ARN:", context.invoked_function_arn)  
    print("Lambda function memory limits in MB:", context.memory_limit_in_mb)  
    print("CloudWatch log stream name:", context.log_stream_name)  
    print("CloudWatch log group name:", context.log_group_name)
```

Lambda – Destinations

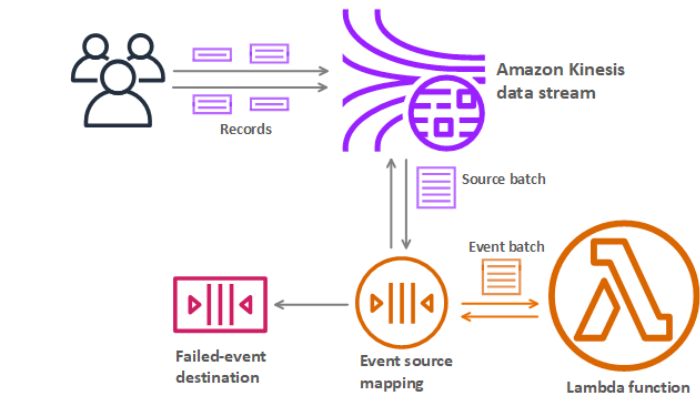
- Nov 2019: Can configure to send result to a destination
- **Asynchronous invocations** - can define destinations for successful and failed event:
 - Amazon SQS
 - Amazon SNS
 - AWS Lambda
 - Amazon EventBridge bus
- *Note: AWS recommends you use destinations instead of DLQ now (but both can be used at the same time)*
- **Event Source mapping:** for discarded event batches
 - Amazon SQS
 - Amazon SNS
- *Note: you can send events to a DLQ directly from SQS*

Destinations for Asynchronous Invocation



<https://docs.aws.amazon.com/lambda/latest/dg/invoke-async.html>

Event Source Mapping with Kinesis Stream



<https://docs.aws.amazon.com/lambda/latest/dg/invoke-eventsourcemapping.html>



Lambda Execution Role (IAM Role)

- Grants the Lambda function permissions to AWS services / resources
- Sample managed policies for Lambda:
 - AWSLambdaBasicExecutionRole – Upload logs to CloudWatch.
 - AWSLambdaKinesisExecutionRole – Read from Kinesis
 - AWSLambdaDynamoDBExecutionRole – Read from DynamoDB Streams
 - AWSLambdaSQSQueueExecutionRole – Read from SQS
 - AWSLambdaVPCAccessExecutionRole – Deploy Lambda function in VPC
 - AWSXRayDaemonWriteAccess – Upload trace data to X-Ray.
- When you use an event source mapping to invoke your function, Lambda uses the execution role to read event data.
- Best practice: create one Lambda Execution Role per function

Lambda Resource Based Policies

- Use resource-based policies to give other accounts and AWS services permission to use your Lambda resources
- Similar to S3 bucket policies for S3 bucket
- An IAM principal can access Lambda:
 - if the IAM policy attached to the principal authorizes it (e.g. user access)
 - OR if the resource-based policy authorizes (e.g. service access)
- When an AWS service like Amazon S3 calls your Lambda function, the resource-based policy gives it access.

Lambda Environment Variables

- Environment variable = key / value pair in “String” form
- Adjust the function behavior without updating code
- The environment variables are available to your code
- Lambda Service adds its own system environment variables as well
- Helpful to store secrets (encrypted by KMS)
- Secrets can be encrypted by the Lambda service key, or your own CMK

Lambda Logging & Monitoring

- CloudWatch Logs:
 - AWS Lambda execution logs are stored in AWS CloudWatch Logs
 - Make sure your AWS Lambda function has an execution role with an IAM policy that authorizes writes to CloudWatch Logs
- CloudWatch Metrics:
 - AWS Lambda metrics are displayed in AWS CloudWatch Metrics
 - Invocations, Durations, Concurrent Executions
 - Error count, Success Rates, Throttles
 - Async Delivery Failures
 - Iterator Age (Kinesis & DynamoDB Streams)

Lambda Tracing with X-Ray

- Enable in Lambda configuration (**Active Tracing**)
- Runs the X-Ray daemon for you
- Use AWS X-Ray SDK in Code
- Ensure Lambda Function has a correct IAM Execution Role
 - The managed policy is called AWSXRayDaemonWriteAccess
- Environment variables to communicate with X-Ray
 - `_X_AMZN_TRACE_ID`: contains the tracing header
 - `AWS_XRAY_CONTEXT_MISSING`: by default, LOG_ERROR
 - `AWS_XRAY_DAEMON_ADDRESS`: the X-Ray Daemon IP_ADDRESS:PORT



Customization At The Edge

- Many modern applications execute some form of the logic at the edge
- **Edge Function:**
 - A code that you write and attach to CloudFront distributions
 - Runs close to your users to minimize latency
- CloudFront provides two types: **CloudFront Functions & Lambda@Edge**
- You don't have to manage any servers, deployed globally
- Use case: customize the CDN content
- Pay only for what you use
- Fully serverless

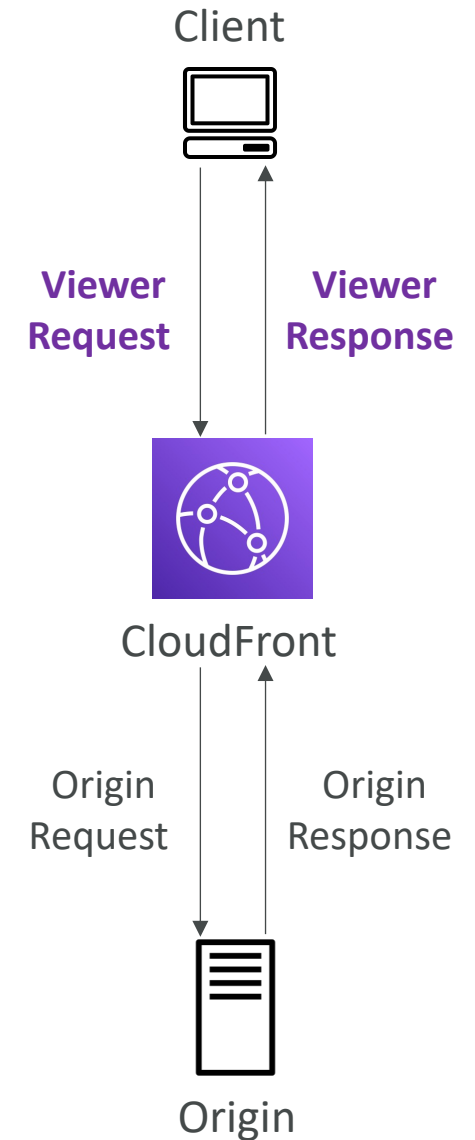
CloudFront Functions & Lambda@Edge Use Cases



- Website Security and Privacy
- Dynamic Web Application at the Edge
- Search Engine Optimization (SEO)
- Intelligently Route Across Origins and Data Centers
- Bot Mitigation at the Edge
- Real-time Image Transformation
- A/B Testing
- User Authentication and Authorization
- User Prioritization
- User Tracking and Analytics

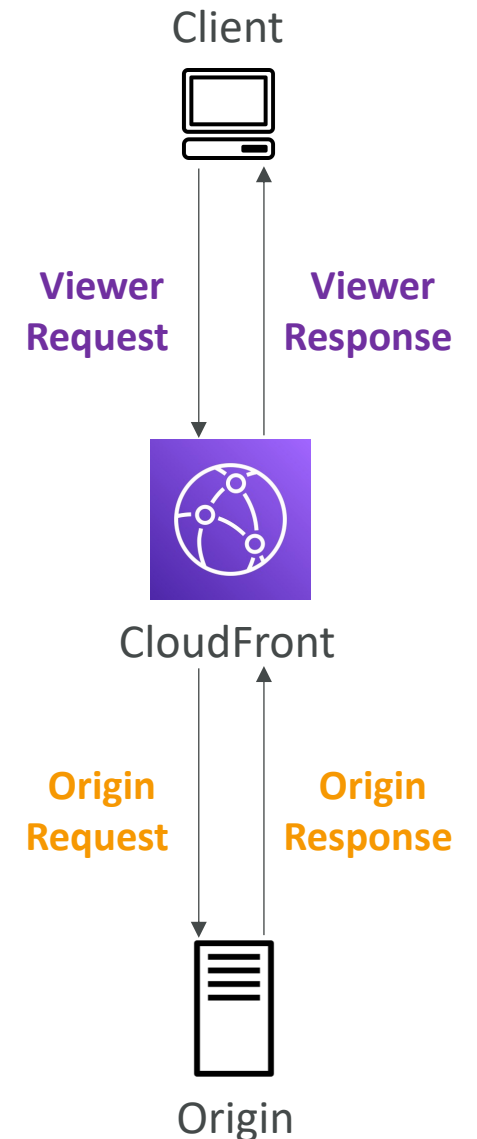
CloudFront Functions

- Lightweight functions written in JavaScript
- For high-scale, latency-sensitive CDN customizations
- Sub-ms startup times, **millions of requests/second**
- Used to change Viewer requests and responses:
 - **Viewer Request:** after CloudFront receives a request from a viewer
 - **Viewer Response:** before CloudFront forwards the response to the viewer
- Native feature of CloudFront (manage code entirely within CloudFront)



Lambda@Edge

- Lambda functions written in NodeJS or Python
- Scales to 1000s of requests/second
- Used to change CloudFront requests and responses:
 - **Viewer Request** – after CloudFront receives a request from a viewer
 - **Origin Request** – before CloudFront forwards the request to the origin
 - **Origin Response** – after CloudFront receives the response from the origin
 - **Viewer Response** – before CloudFront forwards the response to the viewer
- Author your functions in one AWS Region (us-east-1), then CloudFront replicates to its locations



CloudFront Functions vs. Lambda@Edge

	CloudFront Functions	Lambda@Edge
Runtime Support	JavaScript	Node.js, Python
# of Requests	Millions of requests per second	Thousands of requests per second
CloudFront Triggers	- Viewer Request/Response	- Viewer Request/Response - Origin Request/Response
Max. Execution Time	< 1 ms	5 – 10 seconds
Max. Memory	2 MB	128 MB up to 10 GB
Total Package Size	10 KB	1 MB – 50 MB
Network Access, File System Access	No	Yes
Access to the Request Body	No	Yes
Pricing	Free tier available, 1/6 th price of @Edge	No free tier, charged per request & duration

CloudFront Functions vs. Lambda@Edge - Use Cases

CloudFront Functions

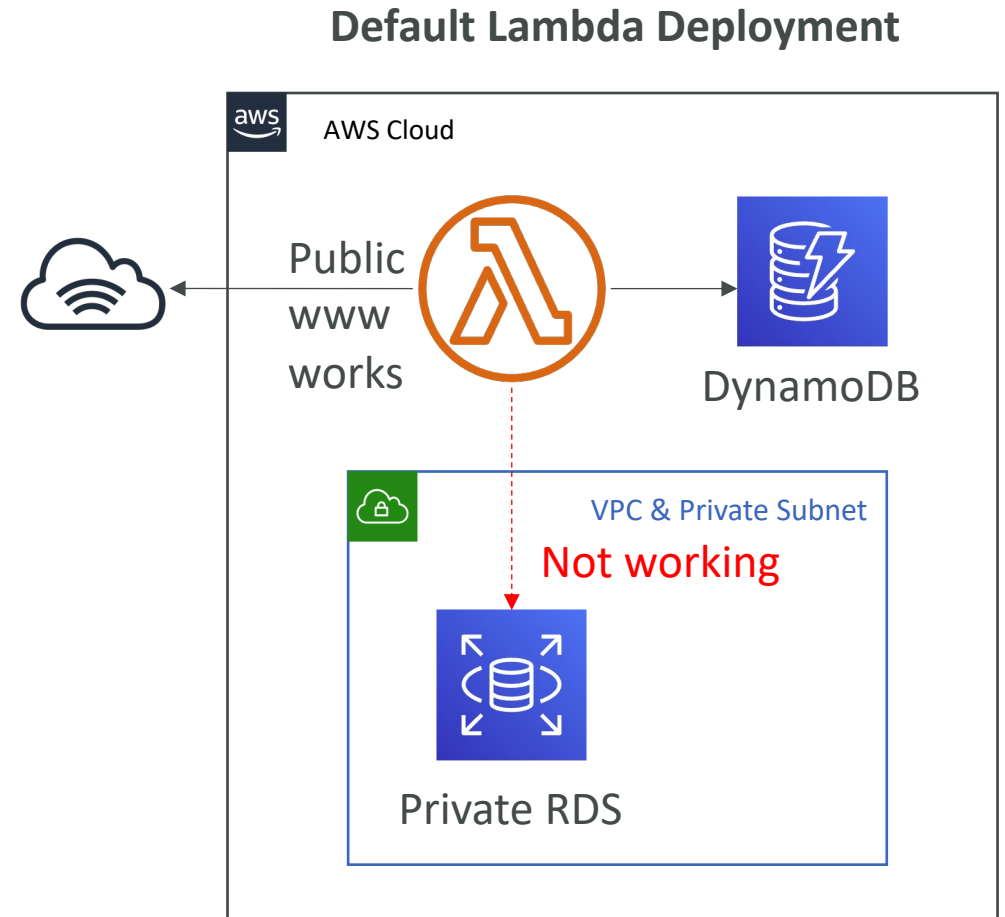
- Cache key normalization
 - Transform request attributes (headers, cookies, query strings, URL) to create an optimal Cache Key
- Header manipulation
 - Insert/modify/delete HTTP headers in the request or response
- URL rewrites or redirects
- Request authentication & authorization
 - Create and validate user-generated tokens (e.g., JWT) to allow/deny requests

Lambda@Edge

- Longer execution time (several ms)
- Adjustable CPU or memory
- Your code depends on a 3rd libraries (e.g., AWS SDK to access other AWS services)
- Network access to use external services for processing
- File system access or access to the body of HTTP requests

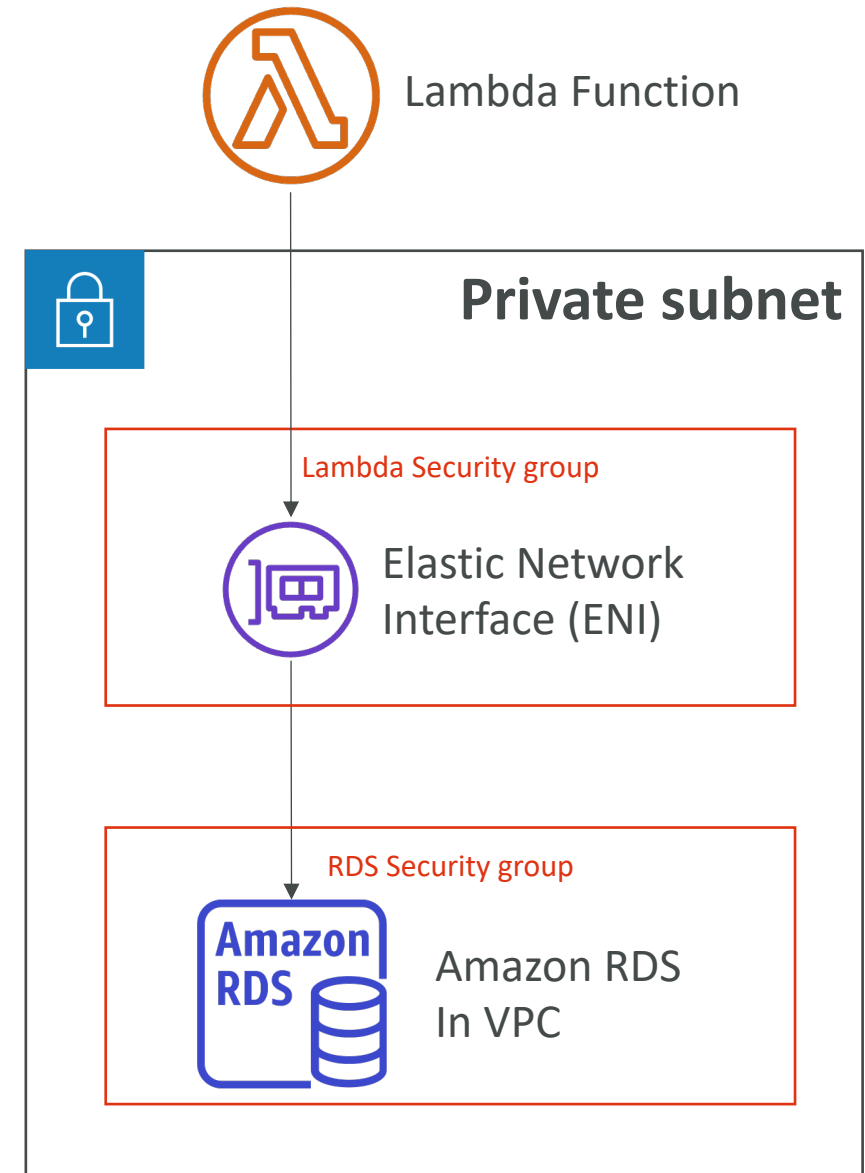
Lambda by default

- By default, your Lambda function is launched outside your own VPC (in an AWS-owned VPC)
- Therefore it cannot access resources in your VPC (RDS, ElastiCache, internal ELB...)



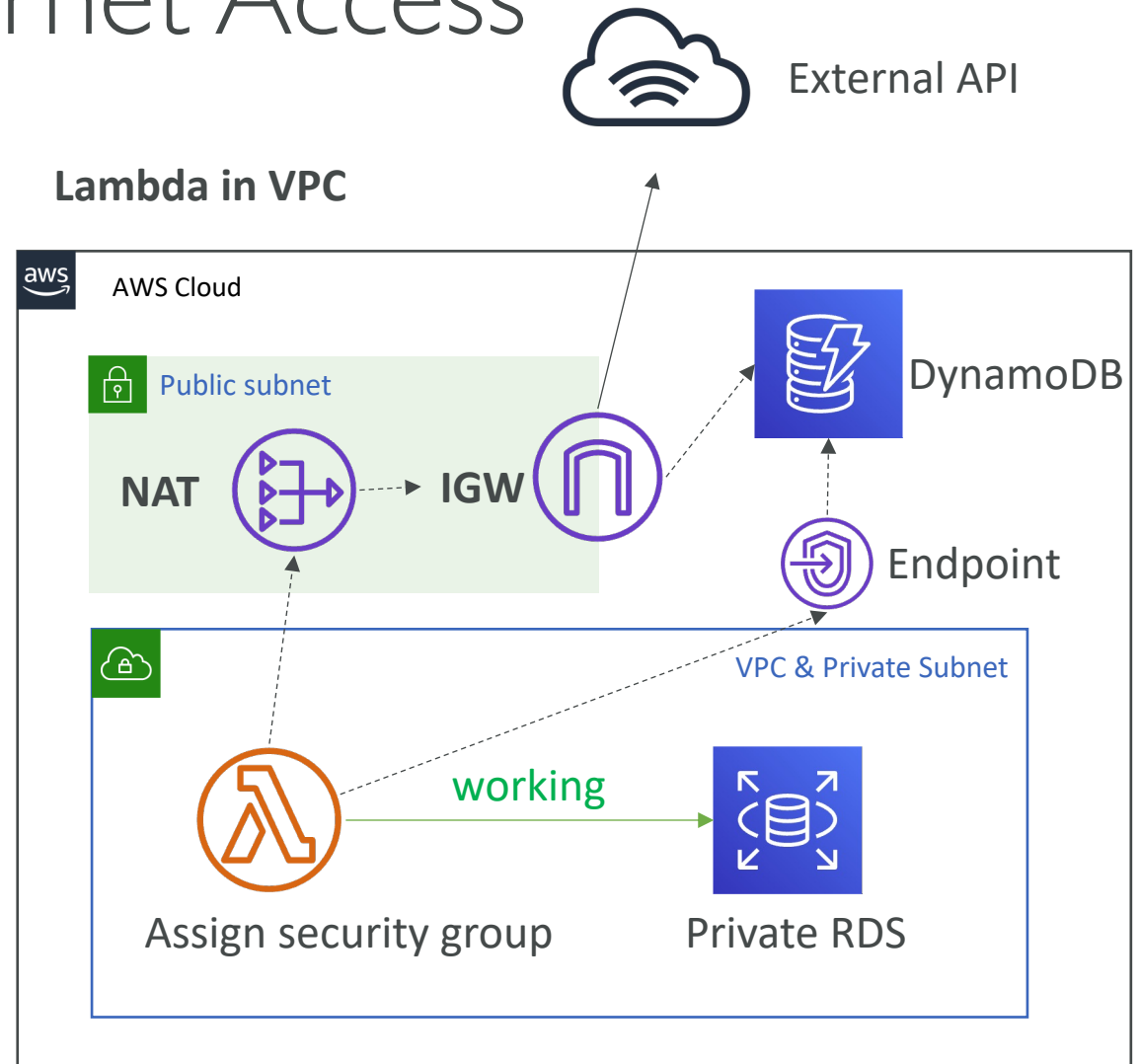
Lambda in VPC

- You must define the VPC ID, the Subnets and the Security Groups
- Lambda will create an ENI (Elastic Network Interface) in your subnets
- `AWSLambdaVPCLambdaAccessExecutionRole`



Lambda in VPC – Internet Access

- A Lambda function in your VPC does not have internet access
- Deploying a Lambda function in a public subnet does not give it internet access or a public IP
- Deploying a Lambda function in a private subnet gives it internet access if you have a **NAT Gateway / Instance**
- You can use **VPC endpoints** to privately access AWS services without a NAT



Note: Lambda - CloudWatch Logs works even without endpoint or NAT Gateway

Lambda Function Configuration

- **RAM:**
 - From 128MB to 10GB in 1MB increments
 - The more RAM you add, the more vCPU credits you get
 - At 1,792 MB, a function has the equivalent of one full vCPU
 - After 1,792 MB, you get more than one CPU, and need to use multi-threading in your code to benefit from it (up to 6 vCPU)
- If your application is CPU-bound (computation heavy), increase RAM
- **Timeout:** default 3 seconds, maximum is 900 seconds (15 minutes)

Lambda Execution Context

- The execution context is a temporary runtime environment that initializes any external dependencies of your lambda code
- Great for database connections, HTTP clients, SDK clients...
- The execution context is maintained for some time in anticipation of another Lambda function invocation
- The next function invocation can “re-use” the context to execution time and save time in initializing connections objects
- The execution context includes the */tmp* directory

Initialize outside the handler

BAD!

```
import os

def get_user_handler(event, context):

    DB_URL = os.getenv("DB_URL")
    db_client = db.connect(DB_URL)
    user = db_client.get(user_id = event["user_id"])

    return user
```

The DB connection is established
At every function invocation

GOOD!

```
import os

DB_URL = os.getenv("DB_URL")
db_client = db.connect(DB_URL)

def get_user_handler(event, context):

    user = db_client.get(user_id = event["user_id"])

    return user
```

The DB connection is established once
And re-used across invocations

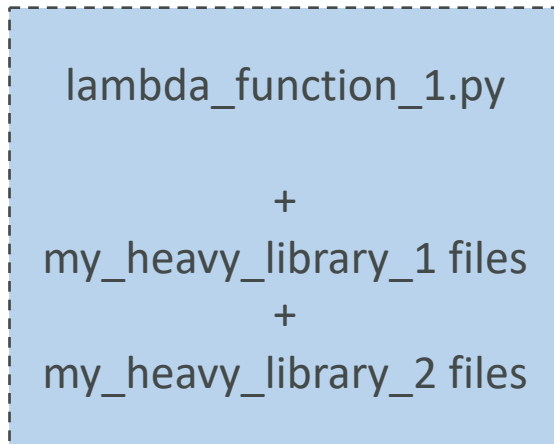
Lambda Functions /tmp space

- If your Lambda function needs to download a big file to work...
- If your Lambda function needs disk space to perform operations...
- You can use the /tmp directory
- Max size is 10GB
- The directory content remains when the execution context is frozen, providing transient cache that can be used for multiple invocations (helpful to checkpoint your work)
- For permanent persistence of object (non temporary), use S3
- To encrypt content on /tmp, you must generate KMS Data Keys

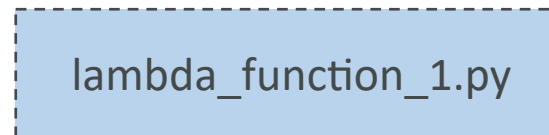
Lambda Layers

- Custom Runtimes
 - Ex: C++ <https://github.com/aws-labs/aws-lambda-cpp>
 - Ex: Rust <https://github.com/aws-labs/aws-lambda-rust-runtime>
- Externalize Dependencies to re-use them:

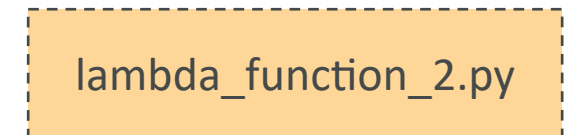
Application Package 1 (30.02MB)



Application Package 1 (20KB)

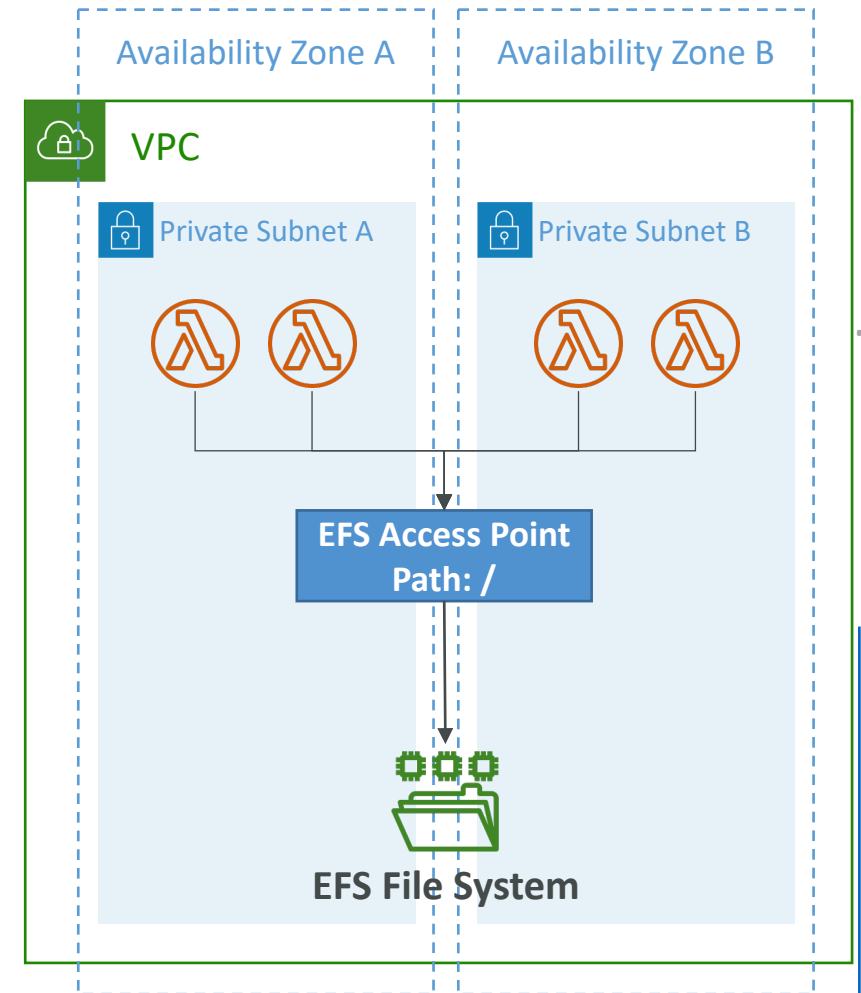


Application Package 1 (60KB)



Lambda – File Systems Mounting

- Lambda functions can access EFS file systems if they are running in a VPC
- Configure Lambda to mount EFS file systems to local directory during initialization
- Must leverage EFS Access Points
- Limitations: watch out for the EFS connection limits (one function instance = one connection) and connection burst limits

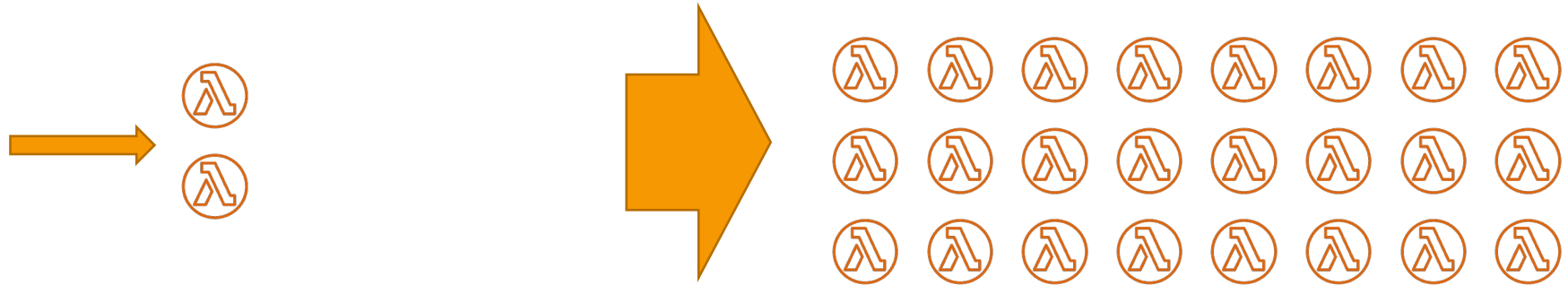


Lambda – Storage Options

	Ephemeral Storage /tmp	Lambda Layers	Amazon S3	Amazon EFS
Max. Size	10,240 MB	5 layers per function up to 250MB total	Elastic	Elastic
Persistence	Ephemeral	Durable	Durable	Durable
Content	Dynamic	Static	Dynamic	Dynamic
Storage Type	File System	Archive	Object	File System
Operations supported	any File System operation	Immutable	Atomic with Versioning	any File System operation
Pricing	Included in Lambda	Included in Lambda	Storage + Requests + Data Transfer	Storage + Data Transfer + Throughput
Sharing/Permissions	Function Only	IAM	IAM	IAM + NFS
Relative Data Access Speed from Lambda	Fastest	Fastest	Fast	Very Fast
Shared Across All Invocations	No	Yes	Yes	Yes

Lambda Concurrency and Throttling

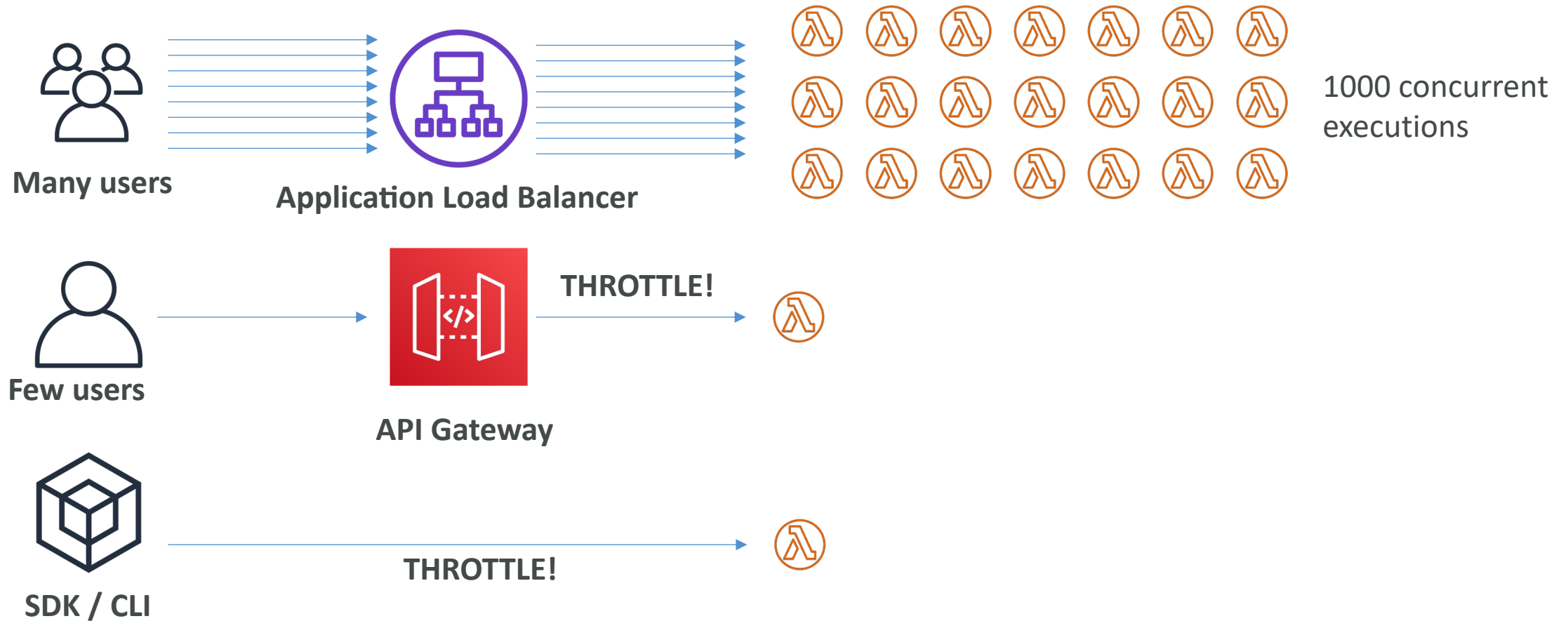
- Concurrency limit: up to 1000 concurrent executions



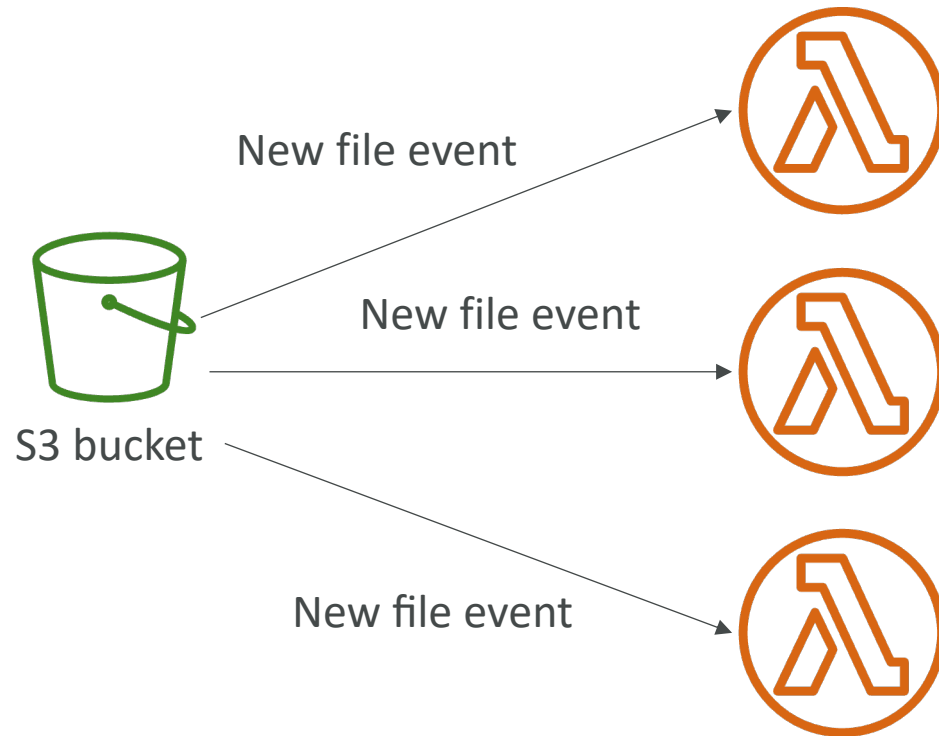
- Can set a “**reserved concurrency**” at the function level (=limit)
- Each invocation over the concurrency limit will trigger a “Throttle”
- Throttle behavior:
 - If synchronous invocation => return `ThrottlerError - 429`
 - If asynchronous invocation => retry automatically and then go to DLQ
- If you need a higher limit, open a support ticket

Lambda Concurrency Issue

- If you don't reserve (=limit) concurrency, the following can happen:



Concurrency and Asynchronous Invocations



- If the function doesn't have enough concurrency available to process all events, additional requests are throttled.
- For throttling errors (429) and system errors (500-series), Lambda returns the event to the queue and attempts to run the function again for up to 6 hours.
- The retry interval increases exponentially from 1 second after the first attempt to a maximum of 5 minutes.

Cold Starts & Provisioned Concurrency

- **Cold Start:**

- New instance => code is loaded and code outside the handler run (init)
- If the init is large (code, dependencies, SDK...) this process can take some time.
- First request served by new instances has higher latency than the rest

- **Provisioned Concurrency:**

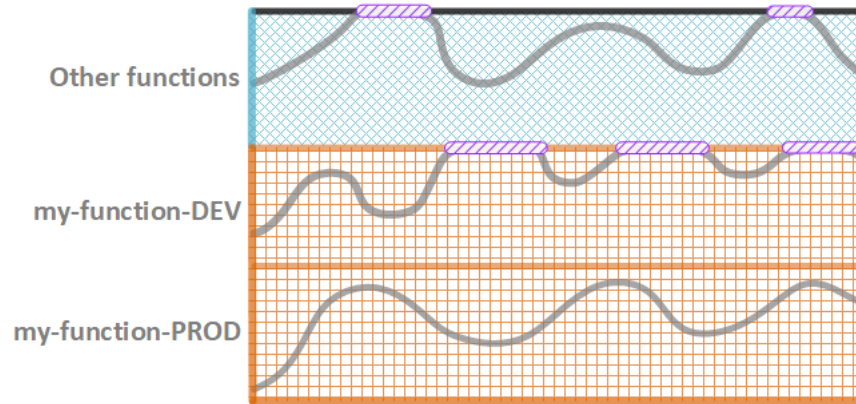
- Concurrency is allocated before the function is invoked (in advance)
- So the cold start never happens and all invocations have low latency
- Application Auto Scaling can manage concurrency (schedule or target utilization)

- **Note:**

- Note: cold starts in VPC have been dramatically reduced in Oct & Nov 2019
- <https://aws.amazon.com/blogs/compute/announcing-improved-vpc-networking-for-aws-lambda-functions/>

Reserved and Provisioned Concurrency

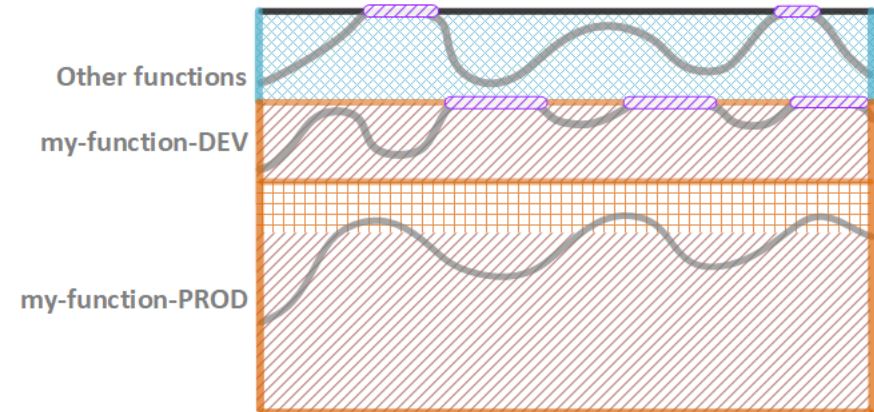
Reserved Concurrency



Legend

-  Function concurrency
-  Reserved concurrency
-  Unreserved concurrency
-  Throttling

Provisioned Concurrency with Reserved Concurrency



Legend

-  Function concurrency
-  Reserved concurrency
-  Provisioned concurrency
-  Unreserved concurrency
-  Throttling

<https://docs.aws.amazon.com/lambda/latest/dg/configuration-concurrency.html>

Lambda Function Dependencies

- If your Lambda function depends on external libraries: for example AWS X-Ray SDK, Database Clients, etc...
- You need to install the packages alongside your code and zip it together
 - For Node.js, use npm & “node_modules” directory
 - For Python, use pip --target options
 - For Java, include the relevant .jar files
- Upload the zip straight to Lambda if less than 50MB, else to S3 first
- Native libraries work: they need to be compiled on Amazon Linux
- AWS SDK comes by default with every Lambda function

Lambda and CloudFormation – inline

```
AWSTemplateFormatVersion: '2010-09-09'
Description: Lambda function inline
Resources:
  primer:
    Type: AWS::Lambda::Function
    Properties:
      Runtime: python3.x
      Role: arn:aws:iam::123456789012:role/lambda-role
      Handler: index.handler
      Code:
        ZipFile: |
          import os

          DB_URL = os.getenv("DB_URL")
          db_client = db.connect(DB_URL)
          def handler(event, context):
            user = db_client.get(user_id = event["user_id"])
            return user
```

- Inline functions are very simple
- Use the `Code.ZipFile` property
- You cannot include function dependencies with inline functions

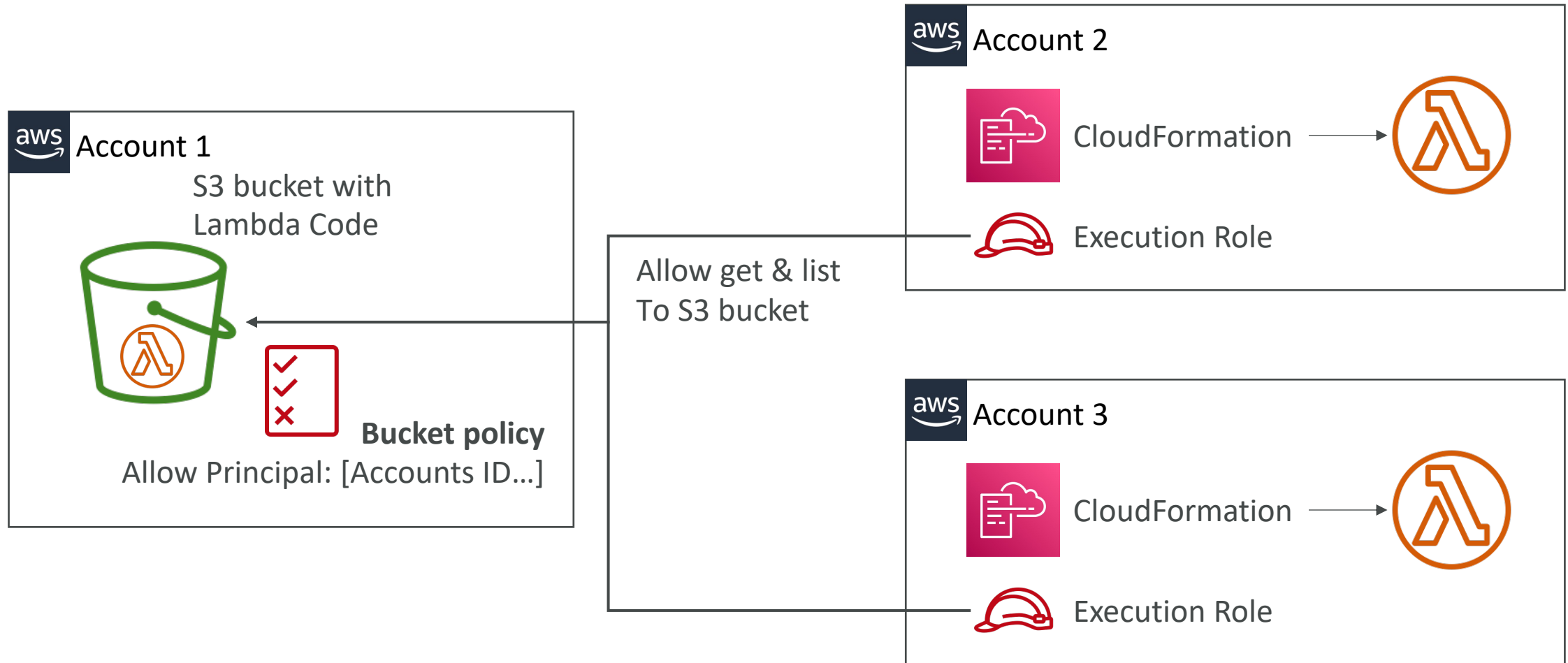
Lambda and CloudFormation – through S3

```
AWSTemplateFormatVersion: '2010-09-09'  
Description: Lambda from S3  
Resources:  
  Function:  
    Type: AWS::Lambda::Function  
    Properties:  
      Handler: index.handler  
      Role: arn:aws:iam::123456789012:role/lambda-role  
      Code:  
        S3Bucket: my-bucket  
        S3Key: function.zip  
        S3ObjectVersion: String  
      Runtime: nodejs12.x
```

- You must store the Lambda zip in S3
- You must refer the S3 zip location in the CloudFormation code
 - S3Bucket
 - S3Key: full path to zip
 - S3ObjectVersion: if versioned bucket
- If you update the code in S3, but don't update S3Bucket, S3Key or S3ObjectVersion, CloudFormation won't update your function

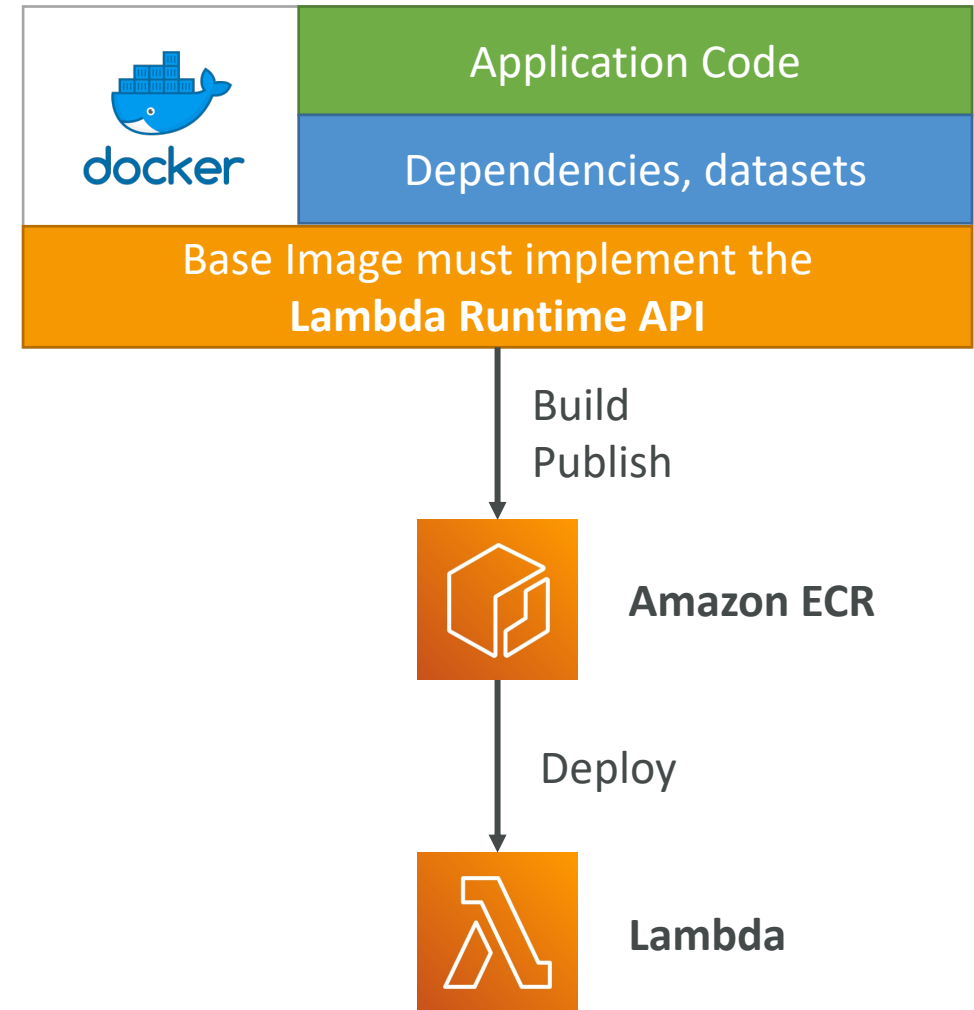
Lambda and CloudFormation – through S3

Multiple accounts



Lambda Container Images

- Deploy Lambda function as container images of up to 10GB from ECR
- Pack complex dependencies, large dependencies in a container
- Base images are available for Python, Node.js, Java, .NET, Go, Ruby
- Can create your own image as long as it implements **the Lambda Runtime API**
- Test the containers locally using the Lambda Runtime Interface Emulator
- Unified workflow to build apps



Lambda Container Images

- Example: build from the base images provided by AWS

```
# Use an image that implements the Lambda Runtime API
FROM amazon/aws-lambda-nodejs:12

# Copy your application code and files
COPY app.js package*.json ./

# Install the dependencies in the container
RUN npm install

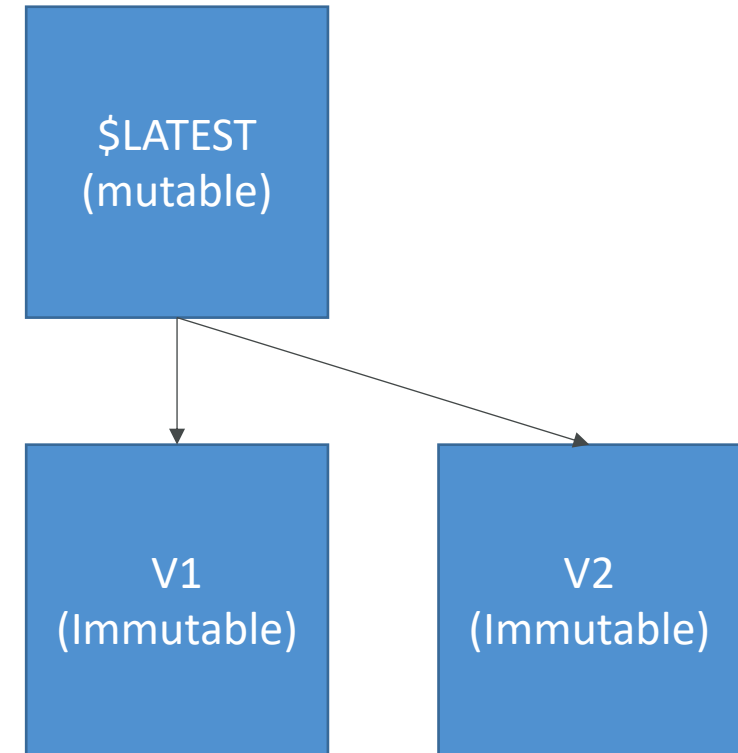
# Function to run when the Lambda function is invoked
CMD [ "app.lambdaHandler" ]
```

Lambda Container Images – Best Practices

- Strategies for optimizing container images:
 - **Use AWS-provided Base Images**
 - Stable, Built on Amazon Linux 2, cached by Lambda service
 - **Use Multi-Stage Builds**
 - Build your code in larger preliminary images, copy only the artifacts you need in your final container image, discard the preliminary steps
 - **Build from Stable to Frequently Changing**
 - Make your most frequently occurring changes as late in your *Dockerfile* as possible
 - **Use a Single Repository for Functions with Large Layers**
 - ECR compares each layer of a container image when it is pushed to avoid uploading and storing duplicates
- Use them to upload large Lambda Functions (up to 10 GB)

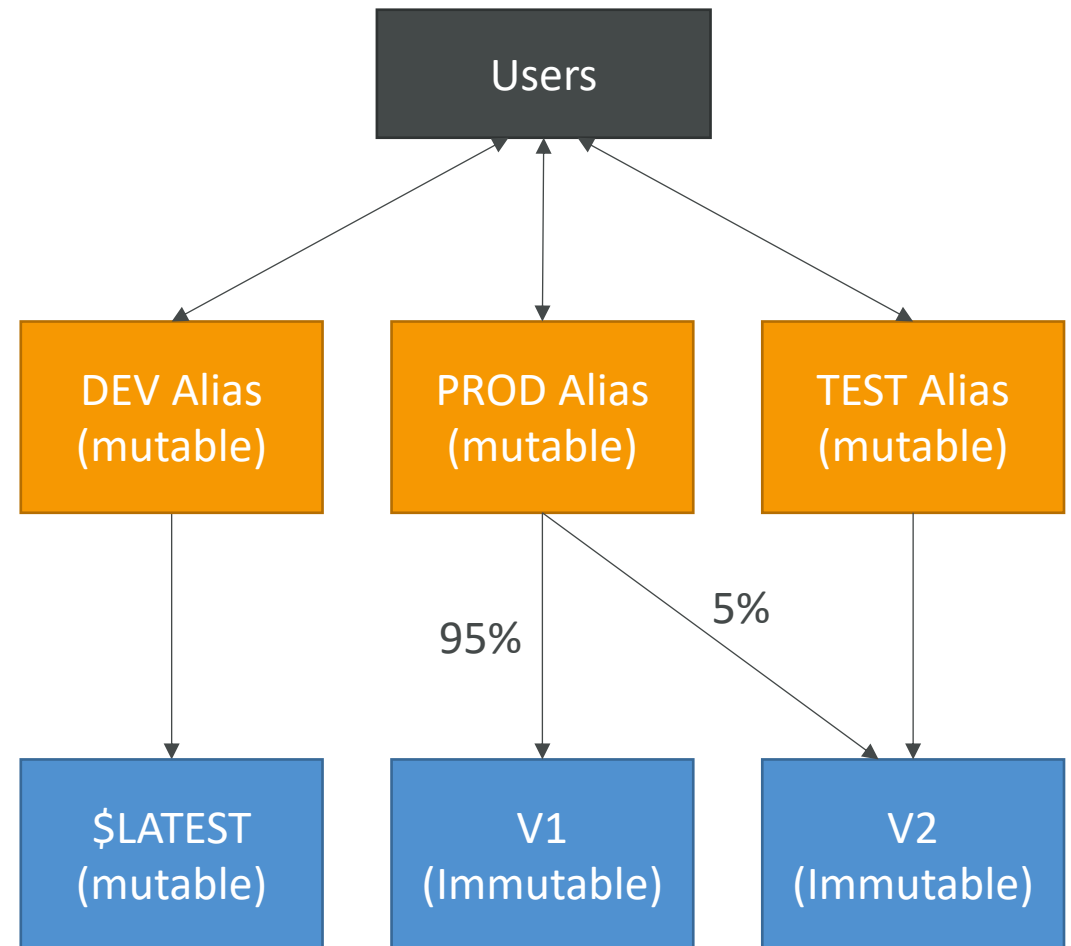
AWS Lambda Versions

- When you work on a Lambda function, we work on **\$LATEST**
- When we're ready to publish a Lambda function, we create a version
- Versions are immutable
- Versions have increasing version numbers
- Versions get their own ARN (Amazon Resource Name)
- Version = code + configuration (nothing can be changed - immutable)
- Each version of the lambda function can be accessed



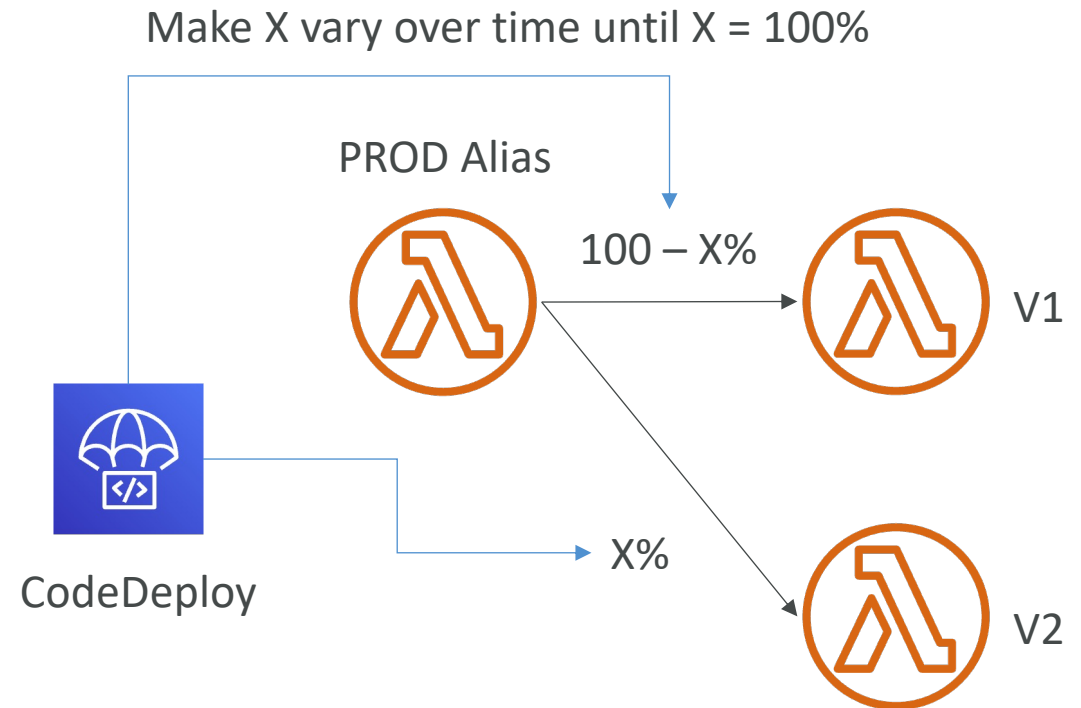
AWS Lambda Aliases

- Aliases are "pointers" to Lambda function versions
- We can define a "dev", "test", "prod" aliases and have them point at different lambda versions
- Aliases are mutable
- Aliases enable Canary deployment by assigning weights to lambda functions
- Aliases enable stable configuration of our event triggers / destinations
- Aliases have their own ARNs
- Aliases cannot reference aliases



Lambda & CodeDeploy

- **CodeDeploy** can help you automate traffic shift for Lambda aliases
- Feature is integrated within the SAM framework
- **Linear:** grow traffic every N minutes until 100%
 - `Linear10PercentEvery3Minutes`
 - `Linear10PercentEvery10Minutes`
- **Canary:** try X percent then 100%
 - `Canary10Percent5Minutes`
 - `Canary10Percent30Minutes`
- **AllAtOnce:** immediate
- Can create Pre & Post Traffic hooks to check the health of the Lambda function



Lambda & CodeDeploy – AppSpec.yml

version: 0.0

Resources:

- myLambdaFunction:

Type: AWS::Lambda::Function

Properties:

Name: myLambdaFunction

Alias: myLambdaFunctionAlias

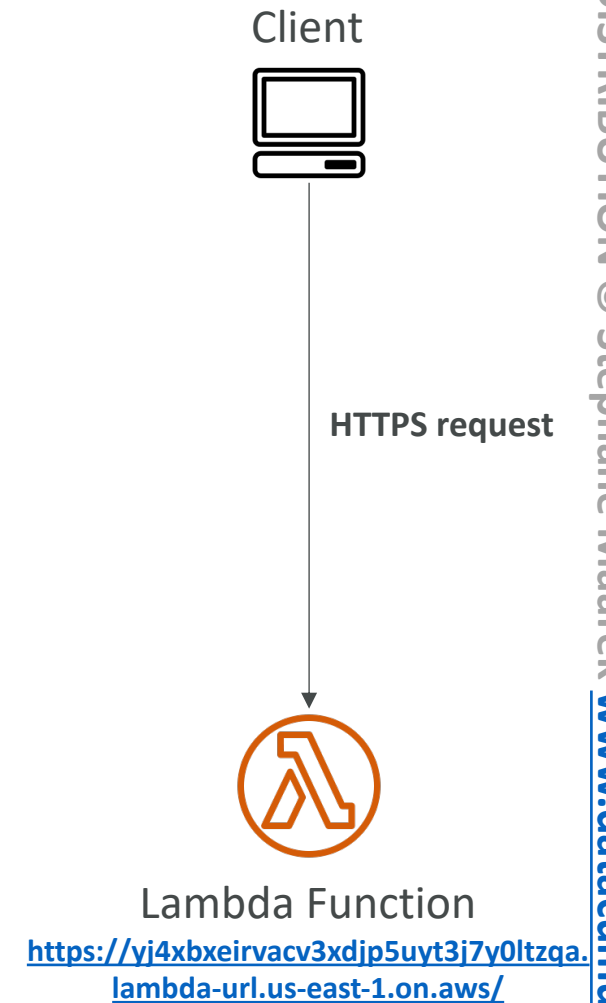
CurrentVersion: 1

TargetVersion: 2

- **Name (required)** – the name of the Lambda function to deploy
- **Alias (required)** – the name of the alias to the Lambda function
- **CurrentVersion (required)** – the version of the Lambda function traffic currently points to
- **TargetVersion (required)** – the version of the Lambda function traffic is shifted to

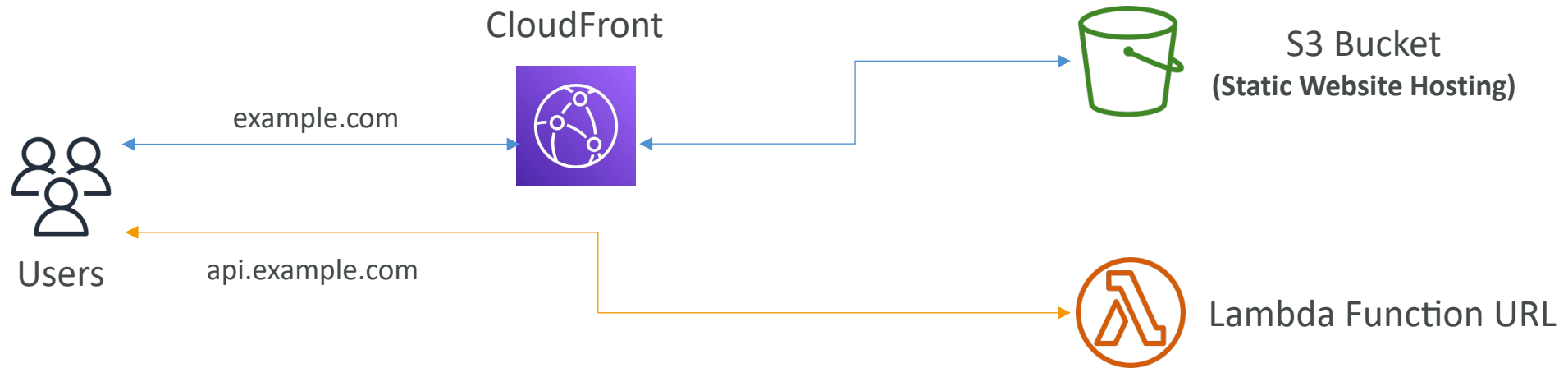
Lambda – Function URL

- Dedicated HTTP(S) endpoint for your Lambda function
- A unique URL endpoint is generated for you (never changes)
 - `https://<url-id>.lambda-url.<region>.on.aws` (dual-stack IPv4 & IPv6)
- Invoke via a web browser, curl, Postman, or any HTTP client
- Access your function URL through the public Internet only
 - Doesn't support PrivateLink (Lambda functions do support)
- Supports Resource-based Policies & CORS configurations
- Can be applied to any function alias or to \$LATEST (can't be applied to other function versions)
- Create and configure using AWS Console or AWS API
- Throttle your function by using Reserved Concurrency



Lambda – Function URL Security

- Resource-based Policy
 - Authorize other accounts / specific CIDR / IAM principals
- Cross-Origin Resource Sharing (CORS)
 - If you call your Lambda function URL from a different domain

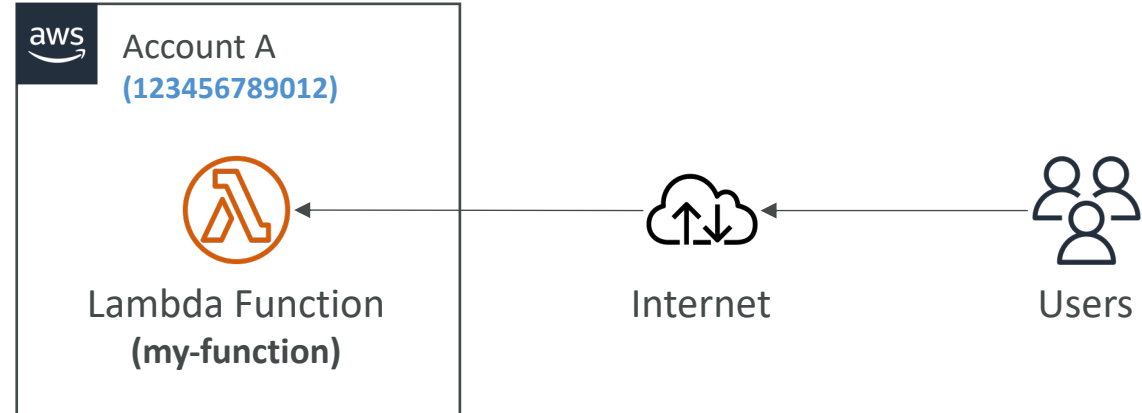


Lambda – Function URL Security

- **AuthType NONE** – allow public and unauthenticated access
 - Resource-based Policy is always in effect (must grant public access)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "lambda:InvokeFunctionUrl",
      "Resource": "arn:aws:lambda:us-east-1:123456789012:
function:my-function",
      "Condition": {
        "StringEquals": {
          "lambda:FunctionUrlAuthType": "NONE"
        }
      }
    }
  ]
}
```

Resource-based Policy

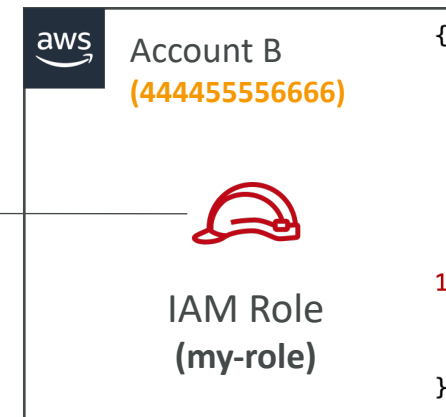
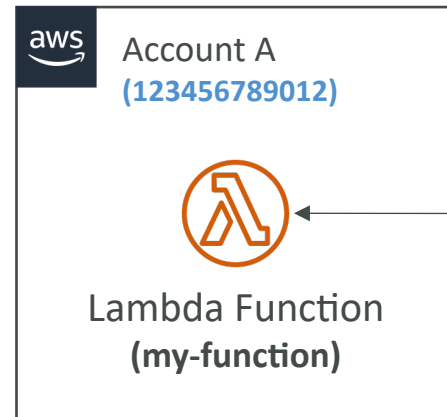


Lambda – Function URL Security

- **AuthType AWS_IAM** – IAM is used to authenticate and authorize requests
 - Both Principal's Identity-based Policy & Resource-based Policy are evaluated
 - Principal must have **lambda:InvokeFunctionUrl** permissions
 - **Same account** – Identity-based Policy **OR** Resource-based Policy as ALLOW
 - **Cross account** – Identity-based Policy **AND** Resource Based Policy as ALLOW

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::444455556666:role/my-role"
      },
      "Action": "lambda:InvokeFunctionUrl",
      "Resource": "arn:aws:lambda:us-east-1:123456789012:
function:my-function",
      "Condition": {
        "StringEquals": {
          "lambda:FunctionUrlAuthType": "AWS_IAM"
        }
      }
    }
  ]
}
```

Resource-based Policy



```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "lambda:InvokeFunctionUrl",
      "Resource": "arn:aws:lambda:us-east-1:
123456789012:function:my-function"
    }
  ]
}
```

Identity-based Policy

AWS Lambda Limits to Know - per region

- **Execution:**

- Memory allocation: 128 MB – 10GB (1 MB increments)
- Maximum execution time: 900 seconds (15 minutes)
- Environment variables (4 KB)
- Disk capacity in the “function container” (in /tmp): 512 MB to 10GB
- Concurrency executions: 1000 (can be increased)

- **Deployment:**

- Lambda function deployment size (compressed .zip): 50 MB
- Size of uncompressed deployment (code + dependencies): 250 MB
- Can use the /tmp directory to load other files at startup
- Size of environment variables: 4 KB



AWS Lambda Best Practices

- Perform heavy-duty work outside of your function handler
 - Connect to databases outside of your function handler
 - Initialize the AWS SDK outside of your function handler
 - Pull in dependencies or datasets outside of your function handler
- Use environment variables for:
 - Database Connection Strings, S3 bucket, etc... don't put these values in your code
 - Passwords, sensitive values... they can be encrypted using KMS
- Minimize your deployment package size to its runtime necessities.
 - Break down the function if need be
 - Remember the AWS Lambda limits
 - Use Layers where necessary
- Avoid using recursive code, never have a Lambda function call itself